

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Разработка виртуальной файловой системы и

реализация файловых систем FAT32 и CDFS

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

А. О. Забанов

(подпись, дата)

(инициалы, фамилия)

Группа ПО-216

Руководитель ВКР

А. А. Чаплыгин

(подпись, дата)

(инициалы, фамилия)

Нормоконтроль

А. А. Чаплыгин

(подпись, дата)

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

А. В. Малышев

(подпись, дата)

(инициалы, фамилия)

Курск 2026 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

«____» _____ 20____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Забанова А. О., шифр 22-06-0157, группа ПО-216

1. Тема «Разработка виртуальной файловой системы и реализация файловых систем FAT32 и CDFS» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1584-с.

2. Срок предоставления работы к защите «09» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

- 1) изучение структуры FAT32 и CDFS;
- 2) проектирование виртуальной файловой системы;
- 3) проектирование интерфейса виртуальной файловой системы;
- 4) реализация виртуальной файловой системы и её интерфейса;
- 5) реализация файловых систем FAT32 и CDFS;
- 6) реализация командного интерпретатора.

3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для программной системы являются: диск с файловой системой.

2) Выходными данными для программной системы являются: выполнение операций над файлами и каталогами.

4. Содержание работы (по разделам):

4.1. Введение.

4.1. Анализ предметной области.

4.2. Техническое задание: основание для разработки, назначение разработки, требования к программной системе, требования к оформлению документации.

4.3. Технический проект: общие сведения о программной системе, проектирование архитектуры программной системы, проектирование интерфейса виртуальной файловой системы.

4.4. Рабочий проект: спецификация компонентов и классов программной системы, тестирование программной системы.

4.5. Заключение.

4.6. Список использованных источников.

5. Перечень графического материала:

Лист 1. Сведения о ВКРБ.

Лист 2. Цель и задачи разработки.

Лист 3. Файловая система FAT32.

Лист 4. Файловая система ISO9660.

Лист 5. Интерфейс виртуальной файловой системы.

Лист 6. Архитектура виртуальной файловой системы.

Лист 7. Диаграмма классов.

Лист 8. Тестирование.

Лист 9. Заключение.

Руководитель ВКР

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

А. О. Забанов

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 131 странице. Работа содержит 34 иллюстрации, 13 таблиц, 52 библиографических источников и 9 листов графического материала. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: файловая система, виртуальная файловая система, FAT32, ISO, CDFS, дескриптор, блок, диск, сектор, файл, каталог, классы, таблица путей, таблица, цепочка, загрузочный сектор, имя файла, расширение файла, данные, байты, биты, запись, массив, Big-Endian, Little-Endian, экстенд, системная область, парсер, исключения, ошибка, методы, фрагментация, наследование, безопасность, журналирование.

Объектом разработки является виртуальная файловая система, поддерживающая файловые системы FAT32 и ISO9660 (CDFS).

Целью выпускной квалификационной работы является создание виртуальной файловой системы, а также изучение структуры FAT32 и CDFS.

В процессе создания виртуальной файловой системы были выделены основные операции над файлами, на основе которых был спроектирован интерфейс виртуальной файловой системы, созданы классы и методы конкретных файловых систем, реализующие интерфейс виртуальной файловой системы. Реализован командный интерпретатор, обеспечивающий интерактивное взаимодействие с виртуальной файловой системой.

При разработке виртуальной файловой системы использовался язык программирования «Common Lisp».

Разработанная виртуальная файловая система была успешно внедрена в «os-ri».

ABSTRACT

The volume of work is 131 page. The work contains 34 illustrations, 13 tables, 52 bibliographic sources and 9 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. Fragments of the source code are presented in annex B.

List of keywords: file system, virtual file system, FAT32, ISO, CDFS, descriptor, block, disk, sector, file, directory, classes, path table, table, chain, boot sector, file name, file extension, data, bytes, bits, record, array, Big-Endian, Little-Endian, extent, system area, parser, exceptions, error, methods, fragmentation, inheritance, security, journaling.

The object of the development is a virtual file system supporting the FAT32 and ISO9660 (CDFS) file systems.

The aim of the graduation thesis is the creation of a virtual file system, as well as studying the structure of FAT32 and CDFS.

In the process of creating the virtual file system, the main file operations were identified, based on which the virtual file system interface was designed, and classes and methods of specific file systems implementing the virtual file system interface were created. A command interpreter was implemented, providing interactive interaction with the virtual file system.

The programming language «Common Lisp» was used in the development of the virtual file system.

The developed virtual file system was successfully implemented in «os-pi».

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	12
1 Анализ предметной области	14
1.1 Файловые системы	14
1.2 Типы файловой системы	15
1.3 История FAT	16
1.4 Устройство FAT32	16
1.4.1 Структура диска с файловой системой FAT32	16
1.4.2 Структура записей в каталогах	18
1.4.3 Преимущества и недостатки FAT32	18
1.5 Устройство CDFS (ISO 9660)	19
1.5.1 Структура диска с файловой системой CDFS	19
1.5.2 Расширения стандарта ISO 9660	20
1.5.3 Преимущества и недостатки CDFS	21
1.6 История ext (Extended File System)	22
1.7 Устройство ext2	22
1.7.1 Структура диска с файловой системой ext2	22
1.7.2 Организация данных файла в ext2	24
1.7.3 Каталог в ext2	24
1.7.4 Преимущества и недостатки ext2	25
2 Техническое задание	26
2.1 Основание для разработки	26
2.2 Цель и назначение разработки	26
2.3 Требования к функциональности	26
2.4 Интерфейс виртуальной файловой системы	27
2.4.1 Загрузка файловой системы	27
2.4.2 Получение пути текущего каталога	27
2.4.3 Просмотр содержимого каталога	28
2.4.4 Создание каталога	29
2.4.5 Удаление каталога	29

2.4.6	Перемещение в каталог	29
2.4.7	Создание файла	29
2.4.8	Удаление файла	30
2.4.9	Открытие файла	30
2.4.10	Закрытие файла	30
2.4.11	Чтение файла	31
2.4.12	Запись в файл	31
2.4.13	Получение позиции в файле	31
2.4.14	Перемещение позиции в файле	32
2.4.15	Проверка на каталог	32
2.4.16	Переименовывание файла или каталога	33
2.4.17	Получение объёма свободного места	33
2.4.18	Изменение атрибутов файла или каталога	33
2.4.19	Получение метаданных файла или каталога	34
2.5	Исключения	35
2.6	Командный интерпретатор	36
2.7	Требования к оформлению документации	36
3	Технический проект	37
3.1	Структура диска	37
3.2	Структура раздела с файловой системой FAT-32	37
3.2.1	Зарезервированная область файловой системы FAT-32	38
3.2.1.1	Блок параметров BIOS	38
3.2.1.2	Структура FSInfo	40
3.2.2	Область таблиц FAT файловой системы FAT-32	40
3.2.2.1	Таблица FAT файловой системы FAT-32	41
3.2.2.2	Определение списка блоков файла по таблице FAT файловой системы FAT-32	42
3.2.3	Область данных файловой системы FAT-32	42
3.2.3.1	Запись в каталоге файловой системы FAT-32	42
3.2.3.2	Запись длинного имени файловой системы FAT-32	44

3.2.3.3 Конвертирование длинного имени в короткое имя файловой системы FAT-32	45
3.2.4 Создание нового файла или каталога файловой системы FAT-32	46
3.2.5 Чтение файла или каталога файловой системы FAT-32	46
3.2.6 Запись в файл файловой системы FAT-32	47
3.3 Структура раздела с файловой системой ISO-9660 (CDFS)	47
3.3.1 Формат данных Little-Endian и Big-Endian файловой системы ISO-9660 (CDFS)	48
3.3.2 Системная область файловой системы ISO-9660 (CDFS)	48
3.3.3 Область дескрипторов тома файловой системы ISO-9660 (CDFS)	48
3.3.3.1 Загрузочная запись дескриптора тома файловой системы ISO-9660 (CDFS)	48
3.3.3.2 Основной дескриптор тома файловой системы ISO-9660 (CDFS)	49
3.3.3.3 Ограничитель набора дескрипторов тома файловой системы ISO-9660 (CDFS)	51
3.3.4 Формат даты и времени в основном дескрипторе тома файловой системы ISO-9660 (CDFS)	51
3.3.5 Таблица путей файловой системы ISO-9660 (CDFS)	52
3.3.6 Область данных файловой системы ISO-9660 (CDFS)	52
3.3.7 Формат даты и времени в записи каталога файловой системы ISO-9660 (CDFS)	54
3.3.8 Создание нового файла или каталога файловой системы ISO-9660 (CDFS)	54
3.3.9 Чтение файла или каталога файловой системы ISO-9660 (CDFS)	55
3.3.10 Запись в файл файловой системы ISO-9660 (CDFS)	55
3.4 Общая архитектура системы	55
3.5 Диаграмма компонентов	56
3.6 Модуль командного интерпретатора	57
3.7 Модуль интерфейса виртуальной файловой системы	58
3.8 Модуль главной загрузочной записи	62

3.9	Модуль виртуальной файловой системы	62
3.10	Модуль файловой системы FAT-32	66
3.11	Модуль файловой системы ISO-9660 (CDFS)	70
3.12	Модуль работы с блоками диска	73
3.13	Модуль драйвера ATA	75
4	Рабочий проект	77
4.1	Модуль «shell.lsp»	77
4.2	Модуль «vfs.lsp»	78
4.3	Модуль «mbr.lsp»	79
4.4	Модули «fat32-class.lsp», «fat32-fat.lsp», «fat32-entry.lsp», «fat32- fs.lsp», «fat32-file.lsp»	80
4.4.1	Класс «FAT32FileSystem»	83
4.4.2	Класс «FAT32File»	85
4.5	Модули «iso9660-fs.lsp», «iso9660-file.lsp»	85
4.5.1	Класс «CDFSFileSystem»	86
4.5.2	Класс «CDFSFile»	88
4.6	Модуль «fs.lsp»	89
4.6.1	Класс «FileSystem»	89
4.6.2	Класс «File»	90
4.7	Модуль «block.lsp»	91
4.8	Модуль «ata.lsp»	91
4.9	Модульное и интеграционное тестирование разработанной вирту- альной файловой системы	92
4.10	Системное тестирование разработанной виртуальной файловой системы	99
	ЗАКЛЮЧЕНИЕ	100
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	101
	ПРИЛОЖЕНИЕ А Представление графического материала	108
	ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	118
	На отдельных листах (CD-RW в прикрепленном конверте)	131

Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)	Лист 1
Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)	Лист 2
Файловая система FAT32 (Графический материал / Файловая система FAT32.png)	Лист 3
Файловая система ISO9660 (Графический материал / Файловая система ISO9660.png)	Лист 4
Интерфейс виртуальной файловой системы (Графический материал / Интерфейс виртуальной файловой системы.png)	Лист 5
Архитектура виртуальной файловой системы (Графический материал / Архитектура виртуальной файловой системы.png)	Лист 6
Диаграмма классов (Графический материал / Диаграмма классов.png)	Лист 7
Тестирование (Графический материал / Тестирование.png)	Лист 8
Заключение (Графический материал / Заключение.png)	Лист 9

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

ВФС (VFS) – виртуальная файловая система (Virtual File System).

ИС – информационная система.

ИТ – информационные технологии.

ПО – программное обеспечение.

РП – рабочий проект.

ТЗ – техническое задание.

ТП – технический проект.

ФС (FS) – файловая система (File System).

ЯП – язык программирования.

ATA – Advanced Technology Attachment (интерфейс доступа к жестким дискам).

BPB – Bios Parameter Block (Блок параметров BIOS)

CDFS – Compact Disc File System (файловая система для оптических дисков).

CL – язык программирования Common Lisp.

FAT – File Allocation Table (таблица размещения файлов).

ISO – International Organization for Standardization (международная организация по стандартизации).

MBR – Master Boot Record (главная загрузочная запись).

ВВЕДЕНИЕ

Виртуальная файловая система (ВФС) представляет собой программный слой абстракции, который предоставляет унифицированный интерфейс для доступа к разнородным физическим и логическим хранилищам данных.

В отличие от традиционных файловых систем, жестко привязанных к конкретному носителю или формату, ВФС оперирует не реальными секторами диска, а логическими объектами, отображая их для пользователя в виде привычной иерархии каталогов и файлов.

Ключевая особенность такой системы заключается в том, что источники данных могут быть географически распределены, иметь различную внутреннюю структуру и управляться независимо друг от друга.

Разработка собственной ВФС позволяет эмулировать поведение стандартных файловых операций – создание, чтение, запись, перемещение и удаление – над объектами, которые физически могут храниться, например, в записях базы данных или в оперативной памяти.

Цель настоящей работы – создание виртуальной файловой системы, а также изучение структуры FAT32 и CDFS. Для достижения поставленной цели необходимо решить *следующие задачи*:

- изучение структуры FAT32 и CDFS;
- проектирование виртуальной файловой системы;
- проектирование интерфейса виртуальной файловой системы;
- реализация виртуальной файловой системы и её интерфейса;
- реализация файловых систем FAT32 и CDFS;
- реализация командного интерпретатора.

Структура и объем работы. Отчет состоит из введения, 4 разделов основной части, заключения, списка использованных источников, 2 приложений. Текст выпускной квалификационной работы равен 131 странице.

Во введении сформулирована цель работы, поставлены задачи разработки, описана структура работы, приведено краткое содержание каждого из разделов.

В первом разделе на стадии описания технической характеристики предметной области приводится сбор информации о файловых системах.

Во втором разделе на стадии технического задания приводятся требования к разрабатываемой виртуальной файловой системе.

В третьем разделе на стадии технического проектирования представлены проектные решения для виртуальной файловой системы.

В четвертом разделе приводится список функций, классов и их методов, использованных при разработке виртуальной файловой системы, производится тестирование разработанной виртуальной файловой системы.

В заключении излагаются основные результаты работы, полученные в ходе разработки.

В приложении А представлен графический материал. В приложении Б представлены фрагменты исходного кода.

1 Анализ предметной области

1.1 Файловые системы

Файловая система – структура, определяющая порядок организации и хранения данных на носителях информации в ЭВМ. Файловые системы управляют хранением файлов, каталогов и метаданных, определяют способ доступа к данным, их размещение и защиту. Без этого данные представляют собой последовательность байтов без структуры.

Основные функции файловой системы включают:

- создание файлов;
- запись в файлы;
- чтение из файлов;
- поиск файлов;
- удаление файлов;
- отслеживание свободного пространства;
- предотвращение конфликтов доступа;
- обеспечение целостности данных.

Файловые системы делятся на:

- дисковые – на носителях информации (FAT32, CDFS, NTFS, ext4, APFS);
- сетевые – сетевой доступ к файловой системе (NFS, SMB);
- виртуальные – абстракция поверх конкретной файловой системы, цель которой обеспечить доступ к разным файловым системам.

Файловая система включает в себя:

1. Блок данных – физический блок информации, в котором хранится файл или его часть. Представляет собой фрагмент данных, который может быть прочитан или записан, чаще всего имеет фиксированный размер.

2. Таблица индексов. Она содержит упорядоченную информацию о расположении блоков данных (индексы), чем ускоряет чтение и поиск файлов.

3. Журналы транзакций – файл, в котором записаны недавние изменения в файловой системе, такие как создание, удаление или изменение файлов, которые помогают обеспечивать целостность данных и восстановить их в случае сбоев.

4. Метаданные – информация о атрибутах файла таких как размер файла, его тип, время создания и изменения, права доступа и другие.

1.2 Типы файловой системы

FAT (File Allocation Table) представляет собой один из первых типов файловой системы. Microsoft создала ее для floppy-дисков. Число после FAT означает сколько бит выделяется под адрес, например FAT12 использовала 12-битные адреса. FAT подходит для небольших дисков, но имеет ограничения в размере.

NTFS (New Technology File System) пришла на смену FAT в Windows NT. Она поддерживает большие объемы, ACL для безопасности и компрессию. NTFS использует MFT (Master File Table) для хранения метаданных.

В Linux чаще всего используются ext2, ext3 (ext2 с журналированием) и ext4 (ext3 с улучшенной производительностью). Эти системы используют inode для описания файлов.

Для macOS Apple использует HFS+ и APFS. APFS добавила snapshots и клонирование файлов.

Сетевые файловые системы включают NFS от Sun Microsystems и CIFS/SMB от Microsoft. Они позволяют совместный доступ к файлам через сеть.

Виртуальные файловые системы, как tmpfs, хранят данные в оперативной памяти.

Файловые системы для флэш-памяти, такие как JFFS или YAFFS, учитывают износ ячеек.

Файловые системы для оптических носителей отличаются. CDFS базируется на ISO 9660. Она предназначена для CD-ROM, где данные записывают один раз.

1.3 История FAT

FAT появилась в 1977 году с Microsoft Disk BASIC. Первая версия использовала 8-битные адреса.

В 1980 году FAT12 стала стандартом для MS-DOS 1.0. Она поддерживала диски до 16 МБ.

FAT16 вышла в 1984 году с MS-DOS 3.0. Она позволяла использовать диски до 2 ГБ. Кластеры имели размер от 2 КБ до 32 КБ.

FAT32 ввели в 1996 году с Windows 95 OSR2. Она поддерживает диски до 2 ТБ. Кластеры начинаются от 4 КБ. FAT32 добавила резервную копию FAT и улучшила управление пространством на диске.

FAT использовали в USB-накопителях и SD-картах. Ее простота обеспечивает совместимость между операционными системами.

1.4 Устройство FAT32

Ключевая концепция FAT32, как и любой FAT, – это таблица размещения файлов (File Allocation Table). Это карта всего дискового пространства, которая показывает, какие кластеры свободны, какие заняты, а какие являются поврежденными. Каждому файлу и каталогу соответствует цепочка кластеров, связи между которыми хранятся в этой таблице.

1.4.1 Структура диска с файловой системой FAT32

Диск с FAT32 делится на 3 основные области:

1. Зарезервированная область (Reserved Region). Эта область начинается с самого первого сектора диска (0 сектор) и содержит критически важную информацию для системы. Главная составляющая – загрузочный сектор (BPB – BIOS Parameter Block). Это первый сектор области. Он содержит служебные данные, необходимые для работы ОС с этой файловой системой. Ключевые поля в BPB:

- 1.1. Размер сектора.

1.2. Количество секторов в одном кластере. Кластер – это минимальная единица размещения файла. Размер кластера зависит от размера тома.

1.3. Количество зарезервированных секторов (включая сам boot-сектор). Для FAT32 обычно равно 32.

1.4. Количество таблиц FAT. Почти всегда 2 (основная и резервная копия для избыточности).

1.5. Размер одной таблицы FAT в секторах.

1.6. Номер первого кластера корневого каталога. В FAT32 корневой каталог может быть любого размера и расположен в области данных, а не в фиксированном месте.

1.7. Сектор в зарезервированной области, где хранится структура FSINFO со служебной информацией (количество свободных кластеров, номер следующего свободного кластера).

1.8. Сектор, где хранится копия загрузочного сектора.

2. Область FAT (FAT Region). Сразу после зарезервированной области идут подряд несколько копий таблицы FAT (их количество указано в BPB). Сама таблица FAT – это массив 32-битных элементов (отсюда и название FAT32). Каждому элементу этого массива соответствует один кластер из области данных.

3. Область данных (Data Region). Это основное пространство для хранения файлов и каталогов, разделённое на кластеры, пронумерованные начиная с 2, потому что кластеры 0 и 1 не существуют, их номера используются для служебных целей в FAT. Корневой каталог это не отдельная область, а обычный каталог, который начинается с кластера, номер которого указан в BPB (Root Cluster). Он может расти и располагаться в любом месте области данных, как и любой другой каталог. Подкаталоги представляют собой специальные файлы, содержащие записи о файлах и других каталогах внутри себя.

1.4.2 Структура записей в каталогах

Каждый файл и каталог в области данных представлен одной или несколькими 32-байтными записями в родительском каталоге.

Стандартная 32-байтная запись содержит:

1. Имя файла (8 байт) и его расширение (3 байта). Для длинных имён файлов перед стандартной записью файла размещаются дополнительные записи, связанные в цепочку. Каждая такая LFN-запись может хранить до 13 символов имени в Unicode.
2. Атрибуты (1 байт), в нём содержатся флаги (например, Read-Only, Hidden, System, Volume Label, Directory, Archive).
3. Резервные поля: время создания (2 байта), дата создания (2 байта), дата последнего доступа (2 байта).
4. Время модификации (2 байта).
5. Дата модификации (2 байта).
6. Номер первого кластера (2 байта high 2 bytes). Старшие 2 байта номера первого кластера.
7. Номер первого кластера (2 байта low 2 bytes). Младшие 2 байта номера первого кластера.
8. Размер файла (4 байта). Размер файла в байтах. Для каталогов обычно указан 0.

1.4.3 Преимущества и недостатки FAT32

Преимущества файловой системы FAT32:

- кроссплатформенность (поддерживается практически всеми операционными системами и устройствами);
- простота (имеет очень простую структуру, что позволяет файловой системе быстро работать на слабых устройствах и встроенных системах);
- эффективность для носителей с малым объёмом (на флеш-накопителях и картах памяти небольшого объема FAT32 показывает очень хорошую скорость работы);

- широкая поддержка инструментами (из-за своей распространенности существует огромное количество утилит для восстановления данных с FAT32-носителей).

Недостатки файловой системы FAT32:

- ограничение на максимальный размер файла (4 ГБ);
- ограничение на максимальный размер тома (раздела);
- отсутствие журналирования;
- низкая отказоустойчивость;
- сильная фрагментация;
- отсутствие современных функций безопасности и разграничения прав;
- неэффективное использование пространства на больших дисках.

1.5 Устройство CDFS (ISO 9660)

CDFS (CD File System) – является реализацией стандарта ISO 9660 для CD-ROM.

ISO 9660 – это международный стандарт, разработанный в 1988 году для обеспечения совместимости компакт-дисков между разными операционными системами и аппаратными платформами. Его основная цель – предоставить простую и универсальную файловую систему для CD-ROM.

1.5.1 Структура диска с файловой системой CDFS

Диск с CDFS делится на 3 основные области:

1. Область дескрипторов тома (Volume Descriptors). Эта область находится в самом начале диска (после системной области начиная с 16-го логического сектора) и описывает всё содержимое диска. Она состоит из последовательности дескрипторов, каждый размером ровно в 1 сектор (2048 байт). Дескрипторы действуют как заголовки для диска. Типы дескрипторов:

- 1.1. Основной дескриптор тома (Primary Volume Descriptor или PVD). Содержит всю базовую информацию о томе: идентификатор системы, идентификатор тома, размер сектора, количество секторов в томе, расположение

корневого каталога (Root Directory Record), расположение таблицы путей (Path Tables) – L-форма и M-форма.

1.2. Ограничитель набора дескрипторов тома (Volume descriptor set terminator). Обозначает конец набора дескрипторов.

2. Таблицы путей (Path Tables). Эта область представляет собой таблицу, содержащую записи каждого каталога на диске. Благодаря этой таблице можно быстро найти местоположение любого каталога. Каждая запись содержит:

- идентификатор каталога;
- номер родительского каталога;
- номер сектора записи каталога в области данных.

Таблица путей хранится в 2 экземплярах: L-Path table (порядок байт от младшего к старшему) и M-Path table (порядок байт от старшего к младшему).

3. Область данных. В этой области хранятся данные файлов и записи каталогов. Каждый файл и подкаталог представлен записью каталогов (Directory Record), который включает:

- размер записи;
- расположение экстента (номер сектора, на котором начинаются данные файла или запись каталога);
- размер файла в байтах;
- атрибуты (например IsDirectory или Hidden);
- длина имени файла;
- имя файла;
- дата и время записи.

В ISO 9660 нет фрагментации, файлы хранятся в виде экстента (последовательной серии секторов), начиная с номера в записи каталога.

1.5.2 Расширения стандарта ISO 9660

ISO 9660 имеет множество ограничений, из-за чего были созданы расширения, сосуществующие с основной структурой:

1. Joliet. Создан Microsoft для поддержки длинных имён файлов в Unicode. Реализовано через создание нового типа дескриптора тома – Supplementary Volume Descriptor, который дополнительно указывает на свою копию корневого каталога и таблиц путей с именами в Unicode. Системы, не поддерживающие Joliet игнорируют SVM, а поскольку Joliet сохраняет файловую систему ISO 9660, никаких проблем с совместимостью не возникает.

2. Rock Ridge Interchange Protocol (RRIP). Создан для поддержки файловых систем UNIX (POSIX): длинные имена файлов, права доступа, идентификаторы пользователя и группы. Реализован через добавления дополнительных полей в запись каталога, которые игнорируется системами без этого расширения.

3. El Torito. Цель – создание загрузочных CD-ROM. Реализовано через создание нового типа дескриптора – Boot Record Descriptor, который содержит информацию, эмулирующую жёсткий диск или флоппи-диск, с которых BIOS может загрузить информацию загрузочного диска.

1.5.3 Преимущества и недостатки CDFS

Преимущества файловой системы CDFS:

- кроссплатформенность (поддерживается практически всеми операционными системами и устройствами);
- простота (для этой файловой системы легко написать драйвер);
- эффективность для последовательного доступа;
- таблицы путей позволяют быстро находить каталоги;
- поддержка расширений, дополняющих ISO 9660.

Недостатки файловой системы CDFS:

- без расширений имеет устаревшие и строгие ограничения;
- только для чтения;
- несмотря на таблицы путей, из-за особенностей CD-приводов произвольный доступ к мелким файлам может быть очень медленным;
- избыточность данных;
- нет шифрования.

1.6 История ext (Extended File System)

Файловая система ext появилась в 1992 году, и является первой файловой системой, специально созданной для ядра Linux.

ext2 вышла в 1993 для решения множества проблем ext.

ext3, вышедшая в 2001 году, добавила в ext2 поддержку журналирования, которое имело 3 режима:

- writeback – сохраняются только метаданные;
- ordered – writeback, но информация об изменении файла происходит после записи в файл;
- journal – сохраняются метаданные файлов и их содержимое.

ext4, созданный в 2008 году, в отличии от ext3:

- увеличил размер одного раздела диска;
- увеличил максимальный размер файла;
- добавил возможность объединять блоки в экстенды при записи файла.

1.7 Устройство ext2

Ключевая концепция ext2 (Second Extended File System) – разделение пространства диска на блоки, которые объединяются в группы блоков. Каждая группа блоков хранит информацию про занятость блоков внутри группы, а также метаданные файлов и их содержимое.

1.7.1 Структура диска с файловой системой ext2

Первый сектор диска с файловой системой ext2 является загрузчиком операционной системы. Далее диск разделён на блоки, а блоки объединены в группы блоков. Каждая группа блоков состоит из нескольких частей:

1. Суперблок (обязательно находится в 0 группе блоков, в следующих группах блоков могут храниться копии). Размер суперблока 1 КБ. Он содержит:

- магическое число – идентификатор файловой системы;

- количество блоков и индексных дескрипторов (inode) в файловой системе;
- количество свободных блоков и индексных дескрипторов в файловой системе;
- размер одного блока;
- размер одного индексного дескриптора;
- количество блоков в группе;
- номер первого блока (принимает значение 0, если размер блока больше 1 КБ, и 1, если равен 1 КБ).

2. Дескриптор группы блоков (обязательно находится в 0 группе блоков, в следующих группах блоков могут храниться копии) – массив, в котором каждый элемент описывает одну группу блоков. Каждый дескриптор группы блоков содержит:

- номер блока, где находится битовая карта занятости блоков группы;
- номер блока, где находится битовая карта занятости индексных дескрипторов этой группы;
- номер блока, где начинается таблица индексных дескрипторов группы;
- количество свободных блоков в группе;
- количество свободных индексных дескрипторов в группе;
- количество каталогов в группе.

3. Битовая карта занятости блоков (обязательно во всех группах блоков). Каждый бит – блок данных в группе. Принимает значение 1, если блок занят, и 0, если свободен.

4. Битовая карта занятости индексных дескрипторов (обязательно во всех группах блоков). Каждый бит – индексный дескриптор в группе. Принимает значение 1, если индексный дескриптор занят, и 0, если свободен.

5. Таблица индексных дескрипторов (обязательно во всех группах блоков) – массив, в котором каждый элемент описывает один файл или каталог. Каждый индексный дескриптор содержит:

- тип файла;

- права доступа;
- идентификатор владельца;
- размер файла;
- время последнего доступа;
- время последней модификации;
- время последнего изменения;
- количество выделенных блоков;
- массив из 15 указателей на блоки, где хранятся данные файла или каталога.

6. Блоки данных (обязательно во всех группах блоков) – последовательность блоков, в которых хранятся данные файлов и каталогов.

1.7.2 Организация данных файла в ext2

В файловой системе ext2, в каждом индексном дескрипторе хранится массив из 15 элементов – многоуровневая схема для эффективного хранения как маленьких, так и больших файлов. Массив содержит:

1. Прямая ссылка. Занимает 12 элементов массива и содержит номера блоков, в которых хранятся данные файла.

2. Косвенная ссылка. Занимает 13 элемент массива и содержит номер блока, который является массивом номеров блоков, в которых хранятся данные файла.

3. Двойная косвенная ссылка. Занимает 14 элемент массива и содержит номер блока, который является массивом косвенных ссылок.

4. Тройная косвенная ссылка. Занимает 15 элемент массива и содержит номер блока, который является массивом двойных косвенных ссылок.

1.7.3 Каталог в ext2

Каталог в ext2 – это файл, содержимое которого представляет собой список записей каталога. Каждая запись каталога содержит:

- длину этой записи каталога;
- номер индексного дескриптора файла;

- длину имени файла;
- массив переменной длины заканчивающийся 0, в котором хранится имя файла.

1.7.4 Преимущества и недостатки ext2

Преимущества файловой системы ext2:

- высокая производительность;
- поддерживается большинством Linux-дистрибутивов;
- контроль доступа (ACL) и расширенных атрибутов файлов по стандарту POSIX;
- эффективность для устройств хранения данных на основе флэш-памяти.

Недостатки файловой системы ext2:

- ограничение размера;
- отсутствие журналирования;
- обязательная проверка (после некорректного завершения работы) на дисках большого объёма может занимать много времени;
- нет шифрования.

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу бакалавра «Разработка виртуальной файловой системы и реализация файловых систем FAT32 и CDFS».

2.2 Цель и назначение разработки

Основной задачей выпускной квалификационной работы является разработка и тестирование виртуальной файловой системы, а также файловых систем FAT32 и CDFS. Для интерактивного взаимодействия с файловой системой необходимо разработать и протестировать командный интерпретатор.

Целью разработки является создание виртуальной файловой системы, а также изучение структуры FAT32 и CDFS.

Задачами данной разработки являются:

- изучение структуры FAT32 и CDFS;
- проектирование виртуальной файловой системы;
- проектирование интерфейса виртуальной файловой системы;
- реализация виртуальной файловой системы и её интерфейса;
- реализация файловых систем FAT32 и CDFS;
- реализация командного интерпретатора.

2.3 Требования к функциональности

Интерфейс виртуальной файловой системы должен включать следующие функции:

- загрузка файловой системы;
- получение пути текущего каталога;
- просмотр содержимого каталога;
- создание каталога;
- удаление каталога;
- перемещение в каталог;

- создание файла;
- удаление файла;
- открытие файла;
- закрытие файла;
- чтение файла;
- запись в файл;
- получение позиции в файле;
- перемещение позиции в файле;
- проверка на каталог;
- переименовывание файла или каталога;
- получение объёма свободного места;
- изменение атрибутов файла или каталога;
- получение метаданных файла или каталога.

2.4 Интерфейс виртуальной файловой системы

2.4.1 Загрузка файловой системы

`load-partition` — загрузить файловую систему из определённого раздела диска. Аргументы: `disk-num`(номер диска, с которого загрузить файловую систему), `part-num`(номер раздела, с которого загрузить файловую систему). Возвращает `nil`. Возможные ошибки: неправильный номер диска, не активный раздел, неизвестный тип файловой системы. Возможные исключения: `argument-error`. Пример:

```
1 ; загрузка файловой системы с первого раздела первого диска
2 (load-partition 0 0)
```

2.4.2 Получение пути текущего каталога

`cur-dir` — получить путь текущего каталога. Аргументов нет. Возвращает полный путь текущего каталога в виде строки. Пример:

```
1 ; полный путь текущего каталога
2 (cur-dir) -> "/dir1/dir2/curentdirectory"
```

Путь можно указывать следующими способами:

```
1 Корневой каталог:
2 "/"
3
4 Каталог относительно корневого каталога:
5 "/directory"
6
7 Текущий каталог:
8 ""
9
10 Каталог относительно текущего каталога:
11 "directory"
12
13 Каждый ".." в пути означает перемещение каталога на уровень выше:
14
15 Каталог на уровень выше от текущего:
16 ".."
17
18 Каталог на два уровня выше от текущего:
19 "../.."
20
21 Каталог в каталоге на уровень выше от текущего:
22 "../dir"
```

2.4.3 Просмотр содержимого каталога

list-dir – просмотреть содержимое каталога. Аргументы: path(путь к каталогу, содержимое которого необходимо вернуть). Возвращает список содержимого каталога, каждый элемент списка – пара, состоящая из символа типа (файл, каталог) и строки названия. Возможные ошибки: ошибка диска. Возможные исключения: path-not-found, not-directory. Пример:

```
1 ; просмотр содержимого корневого каталога
2 (list-dir "/") -> (
3   (file . "file1.txt")
4   (dir . "directory")
5   (file . "file2.txt")
6 )
```

2.4.4 Создание каталога

`create-dir` — создать каталог. Аргументы: `path`(путь к каталогу, который необходимо создать). Возвращает `nil`. Возможные ошибки: ошибка диска. Возможные исключения: `path-not-found`, `not-directory`, `not-enough-space`. Пример:

```
1 ; создать каталог в текущем каталоге
2 (create-dir "directory")
```

2.4.5 Удаление каталога

`remove-dir` — удалить каталог, если он пуст. Аргументы: `path`(путь к каталогу, который нужно удалить). Возвращает `nil`. Возможные ошибки: ошибка диска. Возможные исключения: `path-not-found`, `not-directory`, `not-empty-dir`. Пример:

```
1 ; удалить каталог в корневом каталоге
2 (remove-dir "/directory")
```

2.4.6 Перемещение в каталог

`change-dir` — переместиться в каталог. Аргументы: `path`(путь к каталогу, в который необходимо переместиться). Возможные ошибки: ошибка диска. Возможные исключения: `path-not-found`, `not-directory`. Возвращает `nil`. Пример:

```
1 ; переместиться в каталог относительно текущего каталога
2 (change-dir "directory")
```

2.4.7 Создание файла

`create-file` — создать файл. Аргументы: `path`(путь к файлу, который необходимо создать). Возвращает `nil`. Возможные ошибки: ошибка диска. Возможные исключения: `path-not-found`, `not-directory`, `not-enough-space`. Пример:

```
1 ; создать файл в корневом каталоге
2 (create-file "/file.txt")
```

2.4.8 Удаление файла

`remove-file` — удалить файл. Аргументы: `path`(путь к файлу, который нужно удалить). Возвращает `nil`. Возможные ошибки: ошибка диска. Возможные исключения: `path-not-found`, `not-file`. Пример:

```
1 ; удалить файл в текущем каталоге
2 (remove-file "file.txt")
```

2.4.9 Открытие файла

`open-file` — открыть файл как поток для чтения и записи. Аргументы: `path`(путь к файлу, который нужно открыть). Возвращает объект файла. Возможные ошибки: ошибка диска. Возможные исключения: `path-not-found`, `not-file`. Пример:

```
1 ; открыть файл в корневом каталоге
2 (open-file "/file.txt") -> <объект файла>
```

2.4.10 Заккрытие файла

`close-file` — закрыть файл. Аргументы: `file-object`(объект файла, который необходимо закрыть). Возвращает `nil`. Возможные ошибки: аргумент не является объектом файла. Пример:

```
1 ; открыть файл
2 (setq file (open-file "file.txt"))
3
4 ; закрыть файл
5 (close-file file)
```

2.4.11 Чтение файла

`read-file` — прочитать определённое количество байт из потока файла, и переместить позицию внутри потока на то же количество. Аргументы: `file-object`(объект файла, из которого читать байты), `size`(количество байт, которое необходимо прочитать). Возвращает массив прочитанных байт. Возможные ошибки: ошибка диска, аргумент не является объектом файла. Возможные исключения: `argument-error`, `end-of-file`. Пример:

```
1 ; открыть файл
2 (setq file (open-file "/file.txt"))
3
4 ; прочитать 10 байт из файла
5 (read-file file 10) -> <массив из 10 байт> например #(1 2 3 4 5 6 7 8 9 10)
```

2.4.12 Запись в файл

`write-file` — записать массив байт в поток файла, и переместить позицию внутри потока на то же количество. Аргументы: `file-object`(объект файла, в который необходимо записать массив байт), `buffer`(массив байт, который необходимо записать). Возвращает количество байт, которые было записано. Возможные ошибки: ошибка диска, аргумент не является объектом файла. Возможные исключения: `argument-error`, `not-enough-space`. Пример:

```
1 ; открыть файл
2 (setq file (open-file "file.txt"))
3
4 ; записать 5 байт в файл
5 (write-file file #(1 2 3 4 5)) -> 5
```

2.4.13 Получение позиции в файле

`tell-file` — получить позицию внутри потока файла. Аргументы: `file-object`(объект файла, позицию в котором необходимо вернуть). Возвращает значение позиции в потоке файла. Возможные ошибки: аргумент не является объектом файла. Пример:

```

1 ; открыть файл
2 (setq file (open-file "/file.txt"))
3
4 ; получить значение позиции
5 (tell-file file) -> 0
6
7 ; прочитать 10 байт из файла
8 (read-file file 10)
9
10 ; получить значение позиции
11 (tell-file file) -> 10

```

2.4.14 Перемещение позиции в файле

`seek-file` — переместить позицию внутри потока файла. Аргументы: `file-object`(объект файла, позицию в котором необходимо переместить), `offset`(насколько сместить позицию), `origin`(откуда смещать позицию). Возвращает значение новой позиции в потоке. Возможные ошибки: аргумент не является объектом файла. Возможные исключения: `end-of-file`, `argument-error`. Аргумент `origin` может принимать следующие значения:

- SET — с начала файла;
- END — с конца файла;
- CUR — с текущей позиции.

Пример:

```

1 ; открыть файл
2 (setq file (open-file "file.txt"))
3
4 ; получить значение позиции
5 (tell-file file) -> 0
6
7 ; сместить позицию относительно начала файла на 20
8 (seek-file file 20 'SET)
9
10 ; получить значение позиции
11 (tell-file file) -> 20

```

2.4.15 Проверка на каталог

`is-directory` — проверить, является ли файл каталогом. Аргументы: `path`(путь к файлу, который надо проверить). Возвращает `t`, если файл является

ся каталогом, и nil, если нет. Возможные ошибки: ошибка диска. Возможные исключения: path-not-found. Пример:

```
1 ; проверка на каталог для каталога
2 (is-directory "/directory") -> t
3 (is-directory "/file.txt") -> nil
```

2.4.16 Переименовывание файла или каталога

rename – переименовать файл или каталог. Аргументы: path(путь к файлу или каталогу, который необходимо переименовать), name(новое имя файла или каталога). Возвращает nil. Возможные ошибки: ошибка диска. Возможные исключения: path-not-found. Пример:

```
1 ; создать файл
2 (create-file "" "file.txt")
3
4 ; переименовать файл
5 (rename "file.txt" "exactfile.txt")
```

2.4.17 Получение объёма свободного места

free-space – получить объём свободного места в текущем разделе диска. Аргументов нет. Возвращает размер в байтах. Возможные ошибки: ошибка диска. Пример:

```
1 ; загрузить файловую систему
2 (load-partition 0 0)
3
4 ; получить свободное место
5 (free-space) -> размер свободного места
```

2.4.18 Изменение атрибутов файла или каталога

set-attr – изменить атрибуты файла или каталога. Аргументы: path(путь к файлу или каталогу, чьи атрибуты надо поменять), new-attributes(новые атрибуты файла или каталога). Возвращает nil. Возможные ошибки: ошибка диска. Возможные исключения: path-not-found. Пример:

```
1 ; изменить атрибуты файла  
2 (set-attr "file.txt" '(READ-ONLY))
```

2.4.19 Получение метаданных файла или каталога

`fstat` – получить метаданные файла или каталога. Аргументы: `path`(путь к файлу или каталогу, чьи метаданные надо получить). Возвращает структуру `stat`. Возможные ошибки: ошибка диска. Возможные исключения: `path-not-found`. Структура `stat` состоит из:

1. `name` – название файла.
2. `size` – размер файла в байтах. Для каталогов равно 0.
3. `create-date` – дата создания файла, если нет в конкретной файловой системе, то равно `nil`.
4. `create-time` – время создания файла, если нет в конкретной файловой системе, то равно `nil`.
5. `modify-date` – дата последнего изменения файла, если нет в конкретной файловой системе, то равно `nil`.
6. `modify-time` – время последнего изменения файла, если нет в конкретной файловой системе, то равно `nil`.
7. `access-date` – дата последнего доступа файла, если нет в конкретной файловой системе, то равно `nil`.
8. `access-time` – время последнего изменения файла, если нет в конкретной файловой системе, то равно `nil`.
9. `isdir` – флаг, показывающий, является ли файл каталогом.
10. Другие атрибуты, которые зависят от конкретной файловой системы.

Пример:

```

1 ; получить структуру stat из файла (в файловой системе FAT32)
2 (fstat "/file.txt") -> (
3   (name . "file.txt")
4   (size . 123321)
5   (create-date . (13 9 2025)) ; (d m y)
6   (create-time . (9 40 0)) ; (h m s)
7   (modify-date . (13 09 2025))
8   (modify-time . (9 40 0))
9   (access-date . (13 09 2025))
10  (access-time . nil)
11  (isdir . nil)
12  (create-time-ms . 440) ; миллисекунды времени создания файла
13  (attribute . (Hidden Read-Only)) ; атрибуты файла
14 )
15
16 ; получить структуру stat из файла (в файловой системе CDFS)
17 (fstat "/directory") -> (
18   (name . "directory")
19   (size . 0)
20   (create-date . (13 9 2025)) ; (d m y)
21   (create-time . (9 40 0)) ; (h m s)
22   (modify-date . nil)
23   (modify-time . nil)
24   (access-date . nil)
25   (access-time . nil)
26   (isdir . t)
27   (flags . (Hidden)) ; атрибуты файла
28 )

```

2.5 Исключения

Интерфейс виртуальной файловой системы должен включать следующие исключения:

1. path-not-found — путь не найден.
2. not-directory — по пути найден файл, а не каталог.
3. not-file — по пути найден каталог, а не файл.
4. read-only — файл только для чтения.
5. not-enough-space — не хватает места на диске.
6. end-of-file — чтение за пределами файла.
7. not-empty-dir — попытка удаления непустого каталога.
8. argument-error — некорректные аргументы.
9. file-exists — файл или каталог уже существует.

2.6 Командный интерпретатор

Командный интерпретатор должен включать следующие команды:

1. `clear` – очистить экран. Аргументов нет.
2. `cwd` – получить путь текущего каталога. Аргументов нет.
3. `ls` – получить содержимое текущего каталога. Аргументов нет.
4. `cd` – переместиться в каталог. Аргументы: `path`(путь к каталогу).
5. `cat` – вывести содержимое файла на экран. Аргументы: `path`(путь к файлу).
6. `append` – добавить строку к концу файла. Аргументы: `path`(путь к файлу), `str`(строка, которую необходимо добавить в конец файла).
7. `copy` – скопировать файл. Аргументы: `file1-path`(путь к файлу, который необходимо скопировать), `file2-path`(путь, по которому необходимо создать новый файл (удалить, если уже существует) и скопировать содержимое файла).
8. `mkdir` – создать новый каталог. Аргументы: `path`(путь к каталогу, который необходимо создать).
9. `touch` – создать новый файл. Аргументы: `path`(путь к файлу, который необходимо создать).
10. `rmdir` – удалить каталог, если он пуст. Аргументы: `path`(путь к каталогу, который необходимо удалить, если он пуст).
11. `rm` – удалить файл. Аргументы: `path`(путь к файлу, который необходимо удалить).
12. `chattr` – изменить атрибуты файла. Аргументы: `path`(путь к файлу), `new-attributes`(новые атрибуты файла).

2.7 Требования к оформлению документации

Разработка программной документации и программного изделия должна производиться согласно ГОСТ 19.102-77 и ГОСТ 34.601-90. Единая система программной документации.

3 Технический проект

3.1 Структура диска

Диск имеет следующую структуру:

1. Первый сектор (512 байт) диска занимает главная загрузочная запись (MBR). Главная загрузочная запись содержит следующую информацию:

- код загрузчика (446 байт);
- таблица разделов (4 элемента по 16 байт = 64 байта);
- сигнатура (2 байта). Всегда имеет значение 21930 (55AA₁₆).

Каждая запись раздела из таблицы разделов содержит следующую информацию:

1.1. Признак активности раздела (1 байт), который может принимать только 2 значения: 0 (свободный раздел) или 128 (80₁₆)(раздел занят).

1.2. Адрес начала раздела (3 байта) в виде номеров цилиндра, головки, сектора.

1.3. Код типа файловой системы (1 байт). 0 – пустая запись. 12 (C₁₆) – FAT-32. 148 (94₁₆) – ISO-9660 (CDFS).

1.4. Адрес конца раздела (3 байта) в виде номеров цилиндра, головки, сектора.

1.5. Номер начального сектора LBA (4 байта) раздела.

1.6. Количество секторов LBA (4 байта) раздела.

2. Затем идут сектора четырёх разделов, которые содержат структуры своих файловых систем.

3.2 Структура раздела с файловой системой FAT-32

В записи раздела в таблице разделов главной загрузочной записи устанавливается тип файловой системы = 12 (C₁₆).

Раздел с файловой системой FAT-32 состоит из:

- зарезервированной области;
- области таблиц FAT;
- области данных.

3.2.1 Зарезервированная область файловой системы FAT-32

Зарезервированная область (Reserved Region) содержит следующую информацию:

- 0 сектор раздела – Блок параметров BIOS (BPB – BIOS Parameter Block);
- 1 сектор раздела – структура FSInfo;
- 2-5 сектор раздела – зарезервированные сектора;
- 6 сектор раздела – копия BPB;
- 7 сектор раздела – копия структуры FSInfo;
- далее идут зарезервированные сектора, количество которых указывается в BPB.

3.2.1.1 Блок параметров BIOS

Блок параметров BIOS (BPB) содержит следующую информацию:

1. Команда перехода JMP (3 байта) на загрузчик операционной системы. Может принимать 2 вида значений:

1.1. 1 байт равен 233 ($E9_{16}$), затем 1 байт короткого смещения, а после команда NOP (байт равен 144 (90_{16})).

1.2. 1 байт равен 235 (EB_{16}), затем 2 байта длинного смещения.

Длинное смещение используется, если загрузчик расположен в зарезервированных секторах.

2. Идентификатор версии операционной системы (8 байт) в виде строки.

3. Размер одного сектора (2 байта) в байтах.

4. Размер одного блока (1 байт) в секторах.

5. Количество секторов (2 байта) зарезервированной области.

6. Количество таблиц FAT (1 байт).

7. Количество записей корневого каталога (2 байта). Не используется в файловой системе FAT-32. Имеет значение равное 0.

8. Количество секторов в разделе (2 байта), если количество секторов меньше 65536, иначе равно 0.
9. Медиа дескриптор (1 байт). Хранит значение, определяющее тип диска.
10. Размер таблицы FAT (2 байта) в секторах. Не используется в файловой системе FAT-32. Имеет значение равно 0.
11. Количество секторов в дорожке (2 байта).
12. Количество головок диска (2 байта).
13. Количество скрытых секторов (4 байта) перед началом раздела.
14. Количество секторов в разделе (4 байта), если количество секторов больше или равно 65536.
15. Количество секторов на одну таблицу FAT (4 байта).
16. Номер активной таблицы FAT (2 байта). Побитовое значение:
 - 0-3 бит – номер активной FAT;
 - 4-6 бит – зарезервированы;
 - 7 бит – 0, если все копии FAT копируют активную FAT, иначе 1.
 - 8-15 бит – зарезервированы.
17. Номер версии FAT (2 байта). Старший байт – основной номер версии. Младший байт – младшая версия.
18. Номер первого блока корневого каталога (4 байта) в области данных. Обычно имеет значение равно 2.
19. Номер сектора активной структуры FSInfo (2 байта).
20. Номер сектора копии BPB (2 байта).
21. Зарезервированные байты (12 байт).
22. Номер диска (1 байт).
23. Зарезервированный байт (1 байт).
24. Сигнатура (1 байт) наличия серийного номера или метки раздела. Имеет значение равно 41 (29_{16}), если значение номера или метки раздела не равно 0, иначе имеет значение равно 40 (28_{16}).
25. Серийный номер раздела (4 байта). Обычно имеет значение равно дате и времени при форматировании раздела.

26. Метка раздела (11 байт). Имеет значение равное значению, которое записано в особом файле родительского каталога с атрибутом метки раздела.

27. Строка идентификатора системы (8 байт). Обычно имеет значение "FAT32 "

28. Зарезервированные байты (420 байт).

29. Сигнатура конца (2 байта). Имеет значение 43605 ($AA55_{16}$).

3.2.1.2 Структура FSInfo

Структура FSInfo содержит следующую информацию:

1. Сигнатура структуры FSInfo (4 байта). Имеет значение равное 1096897106 (41615252_{16}).

2. Зарезервированные байты (480 байт).

3. Сигнатура целостности структуры FSInfo (4 байта). Имеет значение равное 1631679090 (61417272_{16}).

4. Текущее количество свободных блоков (4 байта). Если имеет значение равное 4294967295 ($FFFFFFFF_{16}$), то это значение необходимо вычислить.

5. Номер первого свободного блока (4 байта). Указывает на номер блока, с которого необходимо искать свободный блок. Если имеет значение равное 4294967295 ($FFFFFFFF_{16}$), то необходимо искать с начала области данных (2 блок).

6. Зарезервированные байты (12 байт).

7. Сигнатура конца (4 байта). Имеет значение 2857697280 ($AA550000_{16}$).

3.2.2 Область таблиц FAT файловой системы FAT-32

Область FAT (FAT Region) состоит из:

1. Таблицы FAT.

2. Копии таблицы FAT. Количество копий указывается в BPB.

3.2.2.1 Таблица FAT файловой системы FAT-32

Таблица FAT (File Allocation Table) – таблица, представляющая собой последовательность элементов. Каждый элемент представляет собой состояние блока данных в области данных.

Размер таблицы FAT указывается в ВРВ.

Первые два элемента таблицы FAT являются зарезервированными и имеют значения:

1. Первый (0) элемент таблицы FAT имеет значение: старшие 3 байта имеют значение 1048575 ($0FFFF_{16}$), а младший байт имеет значение равное медиа дескриптору из ВРВ.

2. Второй (1) элемент таблицы FAT имеет значение: старшие 2 бита являются флагами для некорректного завершения работы с файловой системой (старший бит – ошибка при размонтировании диска, младший бит – ошибка при чтении или записи).

Каждый элемент таблицы FAT занимает 4 байта (32 бита), из которых старшие 4 бита зарезервированы, и может принимать следующие значения:

1. 0 – блок свободен.
2. От 2 до количества блоков в области данных – блок занят. Значение в таблице – номер следующего блока файла.
3. От количества блоков в области данных + 1 до 268435446 ($FFFFFF_{16}$) – зарезервированные значения и не могут быть использованы.
4. 268435447 ($FFFFFF7_{16}$) – блок повреждён.
5. От 268435448 ($FFFFFF8_{16}$) до 268435454 ($FFFFFFE_{16}$) – зарезервированные значения и не рекомендованы к использованию, но могут быть интерпретированы как то, что блок занят и является последним блоком файла.
6. 268435455 ($FFFFFFF_{16}$) – блок занят и является последним блоком файла.

3.2.2.2 Определение списка блоков файла по таблице FAT файловой системы FAT-32

Зная номер первого блока файла, в таблице FAT находят элемент с номером первого блока файла и смотрят его значение, если блок не является конечным, то смотрят в таблице элемент с этим номером, и так пока не найдётся элемент таблицы со значением конца файла.

В результате получается односвязный список блоков искомого файла.

3.2.3 Область данных файловой системы FAT-32

Область данных (Data Region) – множество блоков, в которых хранятся файлы и каталоги. Нумерация блоков области данных начинается с 2.

Каждый файл и каталог описывается 32-байтной записью. Для родительского каталога нет своей записи, номер первого блока родительского каталога хранится в ВРВ. В родительском каталоге есть файл с особым атрибутом метки раздела.

Каждый каталог, кроме корневого будет иметь 2 файла нулевого размера с названиями "." и "..", которые являются ссылками на текущий и родительский каталог соответственно. Эти файлы будут иметь значение атрибута равным 23 (17_{16}). Значение номера первого блока содержимого будет первым блоком текущего и родительского каталога соответственно.

3.2.3.1 Запись в каталоге файловой системы FAT-32

Каждая 32-байтная запись в каталоге содержит следующую информацию:

1. Короткое имя файла или каталога (11 байт) – названия файла в формате 8+3 (8 байт на имя, 3 байта на расширение). Для файла с атрибутом метки раздела значение имени копируется в поле метки раздела в ВРВ. Допустимые символы в коротком имени:

- цифры;
- заглавные буквы латиницы;

- специальные символы: \$ % ' – _ @ ~ ' ! () { } ^ # &.

Если первый символ короткого имени файла имеет значение 229 ($E5_{16}$), то это значит, что запись в каталоге помечена как удалённая.

2. Атрибуты файла (1 байт). Побитовые атрибуты:

- 0 бит – только для чтения;
- 1 бит – скрытый файл;
- 2 бит – системный файл;
- 3 бит – метка раздела;
- 4 бит – директория;
- 5 бит – изменённый (если файл или каталог был создан, переименован или модифицирован).

3. Зарезервированный байт (1 байт).

4. Сотые доли секунды создания файла или каталога (1 байт). Имеет значение от 0 до 199 (от 0 до 1.99 секунды с шагом 0.01 секунда).

5. Время создания файла или каталога (2 байта). Имеет значение в формате:

- 0-4 бит – секунда делённые на 2;
- 5-10 бит – минута;
- 11-15 – час.

6. Дата создания файла или каталога (2 байта). Имеет значение в формате:

- 0-4 бит – день;
- 5-8 бит – месяц;
- 9-15 – год.

7. Дата последнего доступа к файлу или каталогу (2 байта). Имеет такой же формат как и дата создания.

8. Старшие 2 байта номера первого блока содержимого файла или каталога.

9. Время последнего изменения файла или каталога (2 байта). Имеет такой же формат как и время создания.

10. Дата последнего изменения файла или каталога (2 байта). Имеет такой же формат как и дата создания.

11. Младшие 2 байта номера первого блока содержимого файла или каталога.

12. Размер файла в байтах (4 байта).

3.2.3.2 Запись длинного имени файловой системы FAT-32

Если для имени не хватает места в записи в каталоге, то прямо перед записью в каталоге создаётся запись длинного имени, которая содержит следующую информацию:

1. Номер записи в последовательности записей длинного имени (1 байт). Нумерация должна иметь 7 бит равным 1 (маска 64 (40_{16})) – это позволяет определить, что это запись длинного имени.

2. Первые 5 символов Unicode записи (10 байт).

3. Атрибут (1 байт) равный 16 (F_{16}) – запись длинного имени.

4. Тип длинной записи (1 байт). Для записи длинного имени имеет значение равное 0.

5. Контрольная сумма короткого имени (1 байт).

6. Следующие 6 символов Unicode записи (12 байт).

7. Зарезервированные байты (2 байта).

8. Последние 2 символа Unicode записи (4 байта).

Допустимые символы для длинного имени:

- цифры;
- буквы Unicode;
- пробелы;
- специальные символы: \$ % ' _ @ ~ ' ! () { } ^ # & + , ; = [].

Если имя не полностью помещается в запись длинного имени, то в конце имени ставится терминальный символ (0) и остальные символы ставятся 255 (FF_{16}).

Расположение записей длинного имени перед записью в каталоге идёт в обратном порядке, то есть сначала идёт последняя запись, затем предпоследняя, и так до первой.

3.2.3.3 Конвертирование длинного имени в короткое имя файловой системы FAT-32

Алгоритм конвертации длинного имени в короткое:

1. Конвертирует длинное имя в верхний регистр.
2. Находим аналогичные символы в ASCII, если символа нет, то он заменяется на _.
3. Убираются все пробелы.
4. Убираются все начальные точки.
5. Взять первые 8 (или меньше, если символов основного имени меньше 8) символа основного имени, корректных для короткого имени.
6. Найти последнюю точку в названии перед расширением.
7. Взять первые 3 (или меньше, если символов расширения меньше 3) символа расширения, корректных для короткого имени.

Если хотя бы 1 символ в длинном имени был заменён на _, или длинное имя полностью помещается в короткое имя, или полученное короткое имя уже есть в текущем каталоге, то в конце имени файла добавляется числовой хвост.

Алгоритм добавления числового хвоста:

1. Находится число для числового хвоста – первое доступное число, среди числовых хвостов.
2. В конце основного имени файла добавляется символ ~, а после него число для числового хвоста, если длина основного имени не позволяет добавить числовой хвост, то конец основного имени заменяется на числовой хвост.

Полученное имя является коротким именем конвертированным из длинного имени.

3.2.4 Создание нового файла или каталога файловой системы FAT-32

Порядок действий при создании нового файла или каталога:

1. Высчитывается сколько записей длинного имени понадобится, чтобы вместить имя файла.
2. Создаются записи длинного имени и в каждой устанавливается:
 - номер записи в последовательности;
 - символы имени;
 - атрибут;
3. Внутри содержимого текущего каталога создаётся новая запись в каталоге.
4. Если в блоке текущего каталога нет места, то выделяется новый блок, а в таблице FAT, в элементе с номером последнего блока текущего каталога, устанавливается номер выделенного блока.
5. В созданной записи устанавливается:
 - короткое имя файла и его расширение;
 - если создаётся каталог, то ставится атрибут каталога;
 - устанавливается время и дата создания.
6. Выделяется первый свободный блок, и его номер устанавливается в созданную запись.
7. В таблице FAT, в элемент с номером выделенного для содержимого файла или каталога блока, устанавливается значение конца файла.
8. Если создаётся каталог, то в новом каталоге создаются 2 файла нулевого размера с названиями "." и "..", в которых устанавливается соответствующий атрибут и значение первого блока содержимого.

3.2.5 Чтение файла или каталога файловой системы FAT-32

Порядок действий при чтении файла или каталога:

1. Читается запись в каталоге искомого файла или каталога.

2. Из записи в каталоге узнаётся номер первого блока файла или каталога, а также его размер.

3. Находится нужный блок и читается.

4. В таблице FAT находится элемент с номером блока и читается его значение: если значение элемента не является значением конца файла, то находится номер следующего блока, читается новый блок и находится номер следующего. Процесс продолжается, пока значение элемента таблицы FAT не будет значением конца файла, этот блок будет последним для файла.

3.2.6 Запись в файл файловой системы FAT-32

Порядок действий при записи в файл:

1. Читается запись в каталоге искомого файла.

2. Из записи в каталоге узнаётся номер первого блока файла.

3. Если позиция в которую нужно записывать находится не в этом блоке, то в таблице FAT находится элемент с номером блока и читается его значение: если позиция находится не в этом блоке, то находится номер следующего блока, и так до тех пор пока не найдётся блок с нужной позицией для записи.

4. Начиная с позиции для записи, записываются байты в файл.

5. Если блок закончился, то в таблице FAT находится элемент с номером текущего блока и читается его значение: если значение элемента не является значением конца файла, то находится номер следующего блока, и продолжается запись. Иначе, если значение элемента в таблице имеет значение конца файла, то выделяется новый блок, а в элемент таблицы FAT записывается номер нового блока, и начинается запись в новый блок. Процесс продолжается пока не закончатся свободные блоки, или не закончится запись.

3.3 Структура раздела с файловой системой ISO-9660 (CDFS)

В записи раздела в таблице разделов главной загрузочной записи устанавливается тип файловой системы = 148 (94_{16}).

Раздел с файловой системой ISO-9660 (CDFS) состоит из:

– системной области;

- области дескрипторов тома;
- двух таблиц путей;
- области данных.

3.3.1 Формат данных Little-Endian и Big-Endian файловой системы ISO-9660 (CDFS)

Данные в файловой системе ISO-9660 (CDFS) могут храниться в 3 видах:

1. Little-Endian – порядок байт от младшего к старшему (справа налево).
2. Big-Endian – порядок байт от старшего к младшему (слева направо).
3. Both-Endian – сначала идёт значение в Little-Endian, а после в Big-Endian.

3.3.2 Системная область файловой системы ISO-9660 (CDFS)

Системная область (System Area) занимает первые 64 сектора раздела (32КБ), содержимое которого зарезервировано.

3.3.3 Область дескрипторов тома файловой системы ISO-9660 (CDFS)

Область дескрипторов тома (Volume Descriptors) состоит из:

- загрузочной записи дескриптора тома;
- основного дескриптора тома;
- ограничителя набора дескрипторов тома.

Каждый дескриптор тома занимает 4 сектора (2048 байт).

3.3.3.1 Загрузочная запись дескриптора тома файловой системы ISO-9660 (CDFS)

Загрузочная запись (The Boot Record) содержит следующую информацию:

1. Тип дескриптора тома (1 байт). Имеет значение равное 0.

2. Идентификатор дескриптора тома (5 байт). Имеет значение равное "CD001".

3. Версия дескриптора тома (1 байт). Имеет значение равное 1.

4. Идентификатор загрузочной системы (32 байта). Содержит идентификатор системы, которая может запустить систему из загрузчика. Если не используется, то заполняется пробелами.

5. Идентификатор загрузчика (32 байта).

6. Содержимое загрузчика системы (1977 байт).

3.3.3.2 Основной дескриптор тома файловой системы ISO-9660 (CDFS)

Основной дескриптор тома (The Primary Volume Descriptor) содержит следующую информацию:

1. Тип дескриптора тома (1 байт). Имеет значение равное 1.

2. Идентификатор дескриптора тома (5 байт). Имеет значение равное "CD001".

3. Версия дескриптора тома (1 байт). Имеет значение равное 1.

4. Зарезервированный байт (1 байт).

5. Идентификатор системы (32 байта). Содержит идентификатор системы, которая может взаимодействовать с системной областью. Если не используется, то заполняется пробелами.

6. Идентификатор тома (32 байта). Название раздела.

7. Зарезервированные байты (8 байт).

8. Количество блоков раздела (8 байт). Имеет значение в формате Both-Endian.

9. Зарезервированные байты (32 байта).

10. Размер набора томов. Имеет значение равное 1. Имеет значение в формате Both-Endian.

11. Порядковый номер тома. Имеет значение равное 1. Имеет значение в формате Both-Endian.

12. Размер одного блока в байтах (4 байта). Имеет значение в формате Both-Endian.

13. Размер таблицы путей в байтах (8 байт). Имеет значение в формате Both-Endian.

14. Номер блока, в котором находится таблица путей (4 байта) в формате Little-Endian. Имеет значение в формате Little-Endian.

15. Номер блока, в котором находится копия таблица путей (4 байта) в формате Little-Endian. Имеет значение в формате Little-Endian. Если копии нет, то имеет значение равное 0.

16. Номер блока, в котором находится таблица путей (4 байта) в формате Big-Endian. Имеет значение в формате Big-Endian.

17. Номер блока, в котором находится копия таблица путей (4 байта) в формате Big-Endian. Имеет значение в формате Big-Endian. Если копии нет, то имеет значение равное 0.

18. Запись в каталоге для корневого каталога (34 байта). Имеет длину имени равную 1. Байт имени имеет значение равное 0.

19. Идентификатор набора томов (128 байт). Имеет значение равное идентификатору тома.

20. Идентификатор издателя (128 байт). Если не используется, то заполняется пробелами.

21. Идентификатор поставщика данных (128 байт). Если не используется, то заполняется пробелами.

22. Идентификатор расширения (128 байт). Если не используется, то заполняется пробелами.

23. Идентификатор файла с авторскими правами (37 байт). Если не используется, то заполняется пробелами.

24. Идентификатор файла с аннотацией (37 байт). Если не используется, то заполняется пробелами.

25. Идентификатор библиографического файла (37 байт). Если не используется, то заполняется пробелами.

26. Дата и время создания раздела (17 байт).

27. Дата и время модификации раздела (17 байт).

28. Дата и время истечения срока раздела (17 байт). После этой даты и времени этот раздел нельзя будет использовать. Если даты нет, то имеет значение равное 0.

29. Дата и время активации раздела (17 байт). После этой даты и времени этот раздел можно будет использовать. Если даты нет, то имеет значение равное 0.

30. Версия структуры файлов (1 байт). Имеет значение равное 1.

31. Зарезервированный байт (1 байт).

32. Расширение стандарта ISO-9660 (512 байт).

33. Зарезервированные байты (653 байта).

3.3.3.3 Ограничитель набора дескрипторов тома файловой системы ISO-9660 (CDFS)

Ограничитель набора дескрипторов тома (Volume Descriptor Set Terminator) содержит следующую информацию:

1. Тип дескриптора тома (1 байт). Имеет значение равное 255 (FF_{16}).
2. Идентификатор дескриптора тома (5 байт). Имеет значение равное "CD001".
3. Версия дескриптора тома (1 байт). Имеет значение равное 1.

3.3.4 Формат даты и времени в основном дескрипторе тома файловой системы ISO-9660 (CDFS)

Дата и время в основном дескрипторе тома содержит следующую информацию:

1. Год (4 байта). От 1 до 9999. Имеет значение в виде строки.
2. Месяц (2 байта). От 1 до 12. Имеет значение в виде строки.
3. День (2 байта). От 1 до 31. Имеет значение в виде строки.
4. Час (2 байта). От 0 до 23. Имеет значение в виде строки.
5. Минута (2 байта). От 0 до 59. Имеет значение в виде строки.
6. Секунда (2 байта). От 0 до 59. Имеет значение в виде строки.

7. Сотая секунды (2 байта). От 0 до 99. Имеет значение в виде строки.
8. Смещение часового пояса (1 байт). Исчисляется в 15 минутных интервалах от -48 до 52 ($0 = -48 = -12$ часов, $100 = 52 = +13$ часов). Имеет значение в виде числа.

3.3.5 Таблица путей файловой системы ISO-9660 (CDFS)

Таблица путей (The Path Table) хранится в 2 форматах: Little-Endian и Big-Endian. Разница между ними в том, что значения внутри таблиц хранятся в соответствующем формате. Местоположение таблиц путей указывается в основном дескрипторе тома.

Сама таблица путей – последовательность записей таблицы путей. Каждая запись таблицы путей содержит следующую информацию:

1. Длина имени каталога (1 байт). Для родительского каталога имеет значение равное 1.
2. Длина записи расширенных атрибутов (1 байт). Если не используется, то имеет значение равное 0.
3. Номер блока (4 байта) содержимого каталога.
4. Индекс родительского каталога в таблице путей (2 байта).
5. Имя каталога (длина хранится выше). Для родительского каталога имеет значение равное 0.
6. Если запись таблицы путей имеет нечётную длину, то в конце записи таблицы путей добавляется байт со значением равным 0.

3.3.6 Область данных файловой системы ISO-9660 (CDFS)

Область данных (Data Area) – множество блоков, в которых хранятся файлы и каталоги. Нумерация блоков области данных начинается с 16. Содержимое каталогов описывается последовательностью записей каталога. Для родительского каталога нет своей записи, номер записи таблицы путей корневого каталога, из которой можно узнать его номер блока, хранится в основном дескрипторе тома.

Каждый каталог, кроме корневого будет иметь 2 файла нулевого размера с названиями ""(".") и "\1"(".."), которые являются ссылками на текущий и родительский каталог соответственно. Значение номера первого блока содержимого будет первым блоком текущего и родительского каталога соответственно.

Всё содержимое файла или каталога записывается в виде экстента (непрерывной последовательности блоков).

Каждая запись каталога содержит следующую информацию:

1. Длина записи каталога (1 байт).
2. Длина записи расширенных атрибутов (1 байт). Если не используется, то имеет значение равное 0.
3. Номер блока с содержимым файла или каталога (8 байт). Имеет значение в формате Both-Endian.
4. Размер содержимого в байтах (8 байт). Имеет значение в формате Both-Endian.
5. Дата и время записи (7 байт).
6. Атрибуты файла (1 байт). Побитовые атрибуты:
 - 0 бит – скрытый файл;
 - 1 бит – директория;
 - 2 бит – ассоциативный файл;
 - 3 бит – флаг наличия формата файла в записи расширенных атрибутов;
 - 4 бит – флаг наличия прав доступа в записи расширенных атрибутов;
 - 5 и 6 бит – зарезервированы;
 - 7 бит – не последняя запись каталога для файла (если файл больше 4 ГБ).
7. Размер файлового юнита для пробела между записями (1 байт). Обычно имеет значение равное 0.
8. Разбер промежутка между блоками (1 байт). Обычно имеет значение равное 0.

9. Порядковый номер тома (4 байта). Обычно имеет значение равное 1. Имеет значение в формате Both-Endian.

10. Длина имени файла в байтах (1 байт).

11. Имя файла (длина хранится выше). Разрешённые символы:

- заглавные буквы латиницы;
- специальный символ _.

В конце имени добавляется ";1".

12. Если запись каталога имеет нечётную длину, то в конце записи каталога добавляется байт со значением равным 0.

13. Байты для системного использования расширениями стандарта ISO-9660.

3.3.7 Формат даты и времени в записи каталога файловой системы ISO-9660 (CDFS)

Дата и время в записи каталога (все значения хранятся в виде чисел) содержит следующую информацию:

1. Год (1 байт). Значение года после 1900.
2. Месяц (1 байт). От 1 до 12.
3. День (1 байт). От 1 до 31.
4. Час (1 байт). От 0 до 23.
5. Минута (1 байт). От 0 до 59.
6. Секунда (1 байт). От 0 до 59.
7. Смещение часового пояса (1 байт). Исчисляется в 15 минутных интервалах от -48 до 52.

3.3.8 Создание нового файла или каталога файловой системы ISO-9660 (CDFS)

Файловая система ISO-9660 (CDFS) является файловой системой только для чтения, из-за чего создание нового файла или каталога невозможно.

3.3.9 Чтение файла или каталога файловой системы ISO-9660 (CDFS)

Порядок действий при чтении файла или каталога:

1. Читается запись каталога искомого файла или каталога.
2. Из записи каталога узнаётся номер первого блока файла или каталога, а также его размер.
3. Находится нужный блок и читается.
4. Если блок закончился, то читается следующий блок до тех пор, пока файл или каталог не закончится.

3.3.10 Запись в файл файловой системы ISO-9660 (CDFS)

Файловая система ISO-9660 (CDFS) является файловой системой только для чтения, из-за чего запись в файл невозможна.

3.4 Общая архитектура системы

Система включает в себя следующие модули:

1. Модуль командного интерпретатора. Этот модуль отвечает за командный интерпретатор.
2. Модуль интерфейса виртуальной файловой системы. Этот модуль отвечает за интерфейс виртуальной файловой системы.
3. Модуль главной загрузочной записи. Этот модуль отвечает за загрузку файловой системы из определённого раздела диска.
4. Модуль файловой системы FAT-32. Этот модуль отвечает за реализацию интерфейса файловой системы для файловой системы FAT-32.
5. Модуль файловой системы ISO-9660 (CDFS). Этот модуль отвечает за реализацию интерфейса файловой системы для файловой системы ISO-9660 (CDFS).
6. Модуль виртуальной файловой системы. Этот модуль отвечает за класс абстрактной виртуальной файловой системы и класс абстрактного файла.

7. Модуль работы с блоками диска. Этот модуль отвечает за чтение блоков диска, запись блоков на диск.

8. Модуль драйвера АТА. Этот модуль отвечает за общение с интерфейсом АТА: чтение секторов диска, запись секторов на диск.

3.5 Диаграмма компонентов

На рисунке 3.1 изображена диаграмма компонентов для проектируемой системы. Она включает модули:

- модуль командного интерпретатора;
- модуль интерфейса виртуальной файловой системы;
- модуль главной загрузочной записи;
- модуль файловой системы FAT-32;
- модуль файловой системы ISO-9660 (CDFS);
- модуль виртуальной файловой системы;
- модуль работы с блоками диска;
- модуль драйвера АТА.

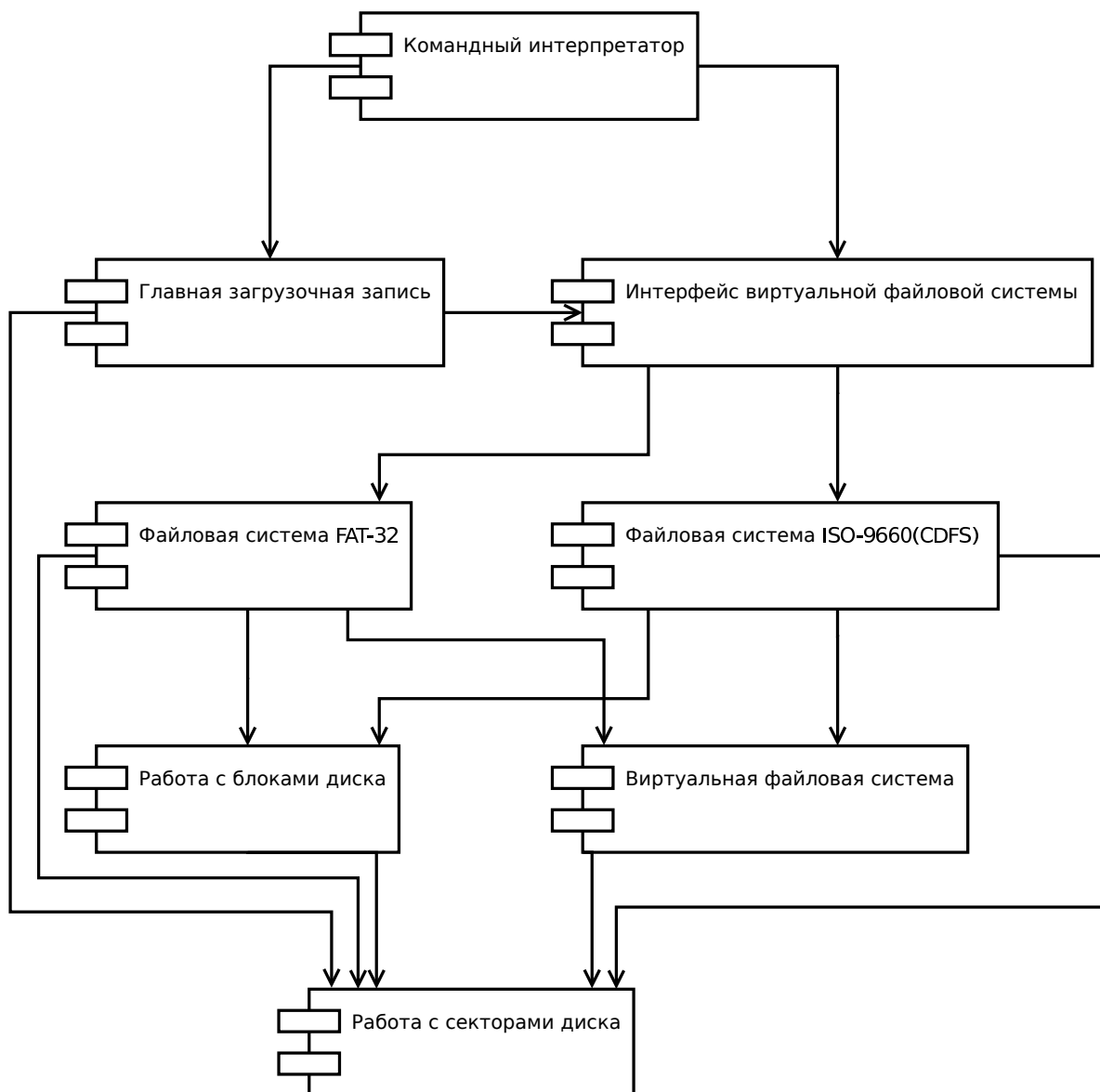


Рисунок 3.1 – Диаграмма компонентов

3.6 Модуль командного интерпретатора

Командный интерпретатор позволяет принимать набранную с клавиатуры строку как команду с аргументами, что даёт возможность интерактивного взаимодействия с файловой системой.

Для реализации командного интерпретатора необходимо:

- создать обработчик нажатия клавиш с клавиатуры;
- создать парсер команд и их аргументов;
- создать обработчики команд с их аргументами.

Модуль командного интерпретатора включает следующие функции:

1. `key-down-handler` – обработчик нажатия клавиш с клавиатуры. Аргументы: `scan`(скан код нажатой клавиши на клавиатуре). Возвращает `nil`.
2. `process-command` – обработка команды. Аргументы: `com`(список введенных символов с клавиатуры). Возвращает `nil`.
3. `parse-commands` – парсер команд. Аргументов нет. Возвращает парсер команд.
4. `parse-command` – парсер определённой команды и её атрибутов. Аргументы: `com`(строка названия команды), `fun`(функция, которую выполняет команда). Возвращает парсер команды и её атрибутов.

3.7 Модуль интерфейса виртуальной файловой системы

Пользователю не нужно знать, какая файловая система находится на разделе диска, поэтому можно выделить функции интерфейса виртуальной файловой системы, а виртуальная файловая система будет вызывать методы той файловой системы, которая находится на диске, для этого нужна функция загрузки файловой системы конкретного типа, которая создаст экземпляр класса файловой системы определённого типа и сохранит его в глобальной переменной `file-system`.

Каждая функция виртуальной файловой системы – вызов соответствующего метода из `file-system`.

Функция открытия файла возвращает экземпляр класса файла конкретной файловой системы, который можно передать в качестве аргументы в функции работы с файлом. Реализация этого экземпляра зависит от конкретной файловой системы.

Пути в стиле UNIX (`/dir/file.txt`).

Структуру каталогов удобно хранить в естественном виде дерева из файлов и каталогов. Это будет файловый объект, хранящий необходимую информацию для того, чтобы писать в этот файл или каталог, у которого есть поле `dir`, которое, если это файл, то равно `nil`, если это нераскрытый каталог, то равно `t`, если это загруженный каталог, то это список файловых объектов. Каталоги раскрываются по мере чтения. Функция `is-directory` возвращает `t` ес-

ли файл – каталог. Глобальная переменная `root-directory` будет хранить файловый объект корневого каталога с деревом файлов и каталогов. Глобальная переменная `working-directory` является ссылкой на часть `root-directory`, хранящая файловый объект текущего каталога. Для раскрытия каталога используется метод виртуальной файловой системы `load-dir`.

Пример дерева файлов и каталогов:

```
1  Дерево корневого каталога:
2  <Файловый объект
3  (dir . (
4      ("File.txt" . <Файловый объект ; обычный файл
5          (dir . nil)>))
6      ("UnopenedDirectory" . <Файловый объект ; нераскрытый каталог
7          (dir . t)>))
8      ("OpenedDirectory" . <Файловый объект ; загруженный каталог
9          (dir . (
10              ("File2.txt" . <Файловый объект
11                  (dir . nil)>))
12              ))>))
13  )>
```

Для того чтобы из строки пути получать дерево каталога требуется функция `load-path`, которая будет при необходимости раскрывать каталоги по пути.

Модуль виртуальной файловой системы включает следующие переменные:

1. `file-system` – экземпляр класса конкретной файловой системы.
2. `root-directory` – дерево файлов и каталогов файловой системы.
3. `working-directory` – дерево текущего каталога.

Модуль виртуальной файловой системы включает следующие функции:

1. `cur-dir` – получить путь текущего каталога. Аргументов нет. Возвращает полный путь текущего каталога в виде строки.
2. `list-dir` – просмотреть содержимое каталога. Аргументы: `path`(путь к каталогу, содержимое которого необходимо вернуть). Возвращает список содержимого каталога, каждый элемент списка – пара, состоящая из символа типа (файл, каталог) и строки названия.

3. `create-dir` — создать каталог. Добавить каталог в дерево файлов и каталогов. Аргументы: `path`(путь к каталогу, в котором создать каталог), `name`(название каталога). Возвращает `nil`.

4. `remove-dir` — удалить каталог, если он пуст. Удалить каталог из дерева файлов и каталогов. Аргументы: `path`(путь к каталогу, который нужно удалить). Возвращает `nil`.

5. `change-dir` — переместиться в каталог. При необходимости раскрыть каталог в дереве файлов и каталогов. Аргументы: `path`(путь к каталогу, в который необходимо переместиться). Возвращает `nil`.

6. `create-file` — создать и открыть файл. Добавить файл в дерево файлов и каталогов. Аргументы: `path`(путь к каталогу, в котором необходимо создать файл), `name`(название файла). Возвращает объект файла.

7. `remove-file` — удалить файл. Удалить файл в дереве файлов и каталогов. Аргументы: `path`(путь к файлу, который нужно удалить). Возвращает `nil`.

8. `open-file` — открыть файл как поток для чтения и записи. При необходимости раскрыть каталоги по пути к файлу в дереве файлов и каталогов. Аргументы: `path`(путь к файлу, который нужно открыть). Возвращает объект файла.

9. `close-file` — закрыть файл. Аргументы: `file-object`(объект файла, который необходимо закрыть). Возвращает `nil`.

10. `read-file` — прочитать определённое количество байт из потока файла, и переместить позицию внутри потока на то же количество. Аргументы: `file-object`(объект файла, из которого читать байты), `size`(количество байт, которое необходимо прочитать). Возвращает массив прочитанных байт.

11. `write-file` — записать массив байт в поток файла, и переместить позицию внутри потока на то же количество. Аргументы: `file-object`(объект файла, в который необходимо записать массив байт), `buffer`(массив байт, который необходимо записать). Возвращает количество байт, которые было записано.

12. `tell-file` — получить позицию внутри потока файла. Аргументы: `file-object`(объект файла, позицию в котором необходимо вернуть).

13. `seek-file` – переместить позицию внутри потока файла. Аргументы: `file-object`(объект файла, позицию в котором необходимо переместить), `offset`(насколько сместить позицию), `origin`(откуда смещать позицию). Возвращает значение новой позиции в потоке.

14. `is-directory` – проверить, является ли файл каталогом. Аргументы: `path`(путь к файлу, который надо проверить). Возвращает `t`, если файл является каталогом, и `nil`, если нет.

15. `rename` – переименовать файл или каталог. Аргументы: `path`(путь к файлу или каталогу, который необходимо переименовать), `name`(новое имя файла или каталога). Возвращает `nil`.

16. `free-space` – получить объём свободного места в текущем разделе диска. Аргументов нет. Возвращает размер в байтах.

17. `set-attr` – изменить атрибуты файла или каталога. Аргументы: `path`(путь к файлу или каталогу, чьи атрибуты необходимо поменять), `new-attributes`(новые атрибуты файла или каталога). Возвращает `nil`.

18. `fstat` – получить метаданные файла или каталога. Аргументы: `path`(путь к файлу или каталогу, чьи метаданные надо получить). Возвращает структуру `stat`. Структура `stat` состоит из:

18.1. `name` – название файла;

18.2. `size` – размер файла в байтах. Для каталогов равно 0;

18.3. `create-date` – дата создания файла, если нет в конкретной файловой системе, то равно `nil`;

18.4. `create-time` – время создания файла, если нет в конкретной файловой системе, то равно `nil`;

18.5. `modify-date` – дата последнего изменения файла, если нет в конкретной файловой системе, то равно `nil`;

18.6. `modify-time` – время последнего изменения файла, если нет в конкретной файловой системе, то равно `nil`;

18.7. `access-date` – дата последнего доступа файла, если нет в конкретной файловой системе, то равно `nil`;

18.8. `access-time` – время последнего изменения файла, если нет в конкретной файловой системе, то равно `nil`;

18.9. `dir` – флаг, показывающий, является ли файл каталогом;

18.10. другие атрибуты, которые зависят от конкретной файловой системы.

19. `load-path` – получить дерево каталога по абсолютному или относительному пути. При необходимости раскрыть каталоги по пути к каталогу в дереве файлов и каталогов. Аргументы: `path`(путь к каталогу). Возвращает дерево каталога по пути.

3.8 Модуль главной загрузочной записи

Для того, чтобы загрузить файловую систему из определённого раздела, необходимо найти нужную запись раздела из таблицы разделов, получить номер начального сектора и количество секторов и вызвать функцию загрузки файловой системы определённого типа из модуля виртуальной файловой системы.

Модуль главной загрузочной записи включает следующую функцию:

`load-partition` – загрузить файловую систему из определённого раздела диска. Аргументы: `disk-num`(номер диска, с которого загрузить файловую систему), `part-num`(номер раздела, с которого загрузить файловую систему). Возвращает `nil`.

3.9 Модуль виртуальной файловой системы

Виртуальная файловая система – два класса, методы которого являются функциями интерфейса виртуальной файловой системы.

Поскольку функции `open-file`, `close-file`, `read-file`, `write-file`, `seek-file`, `tell-file` работают с файлами, то функция `open-file` будет возвращать файловый объект, а остальные будут работать с этим файловым объектом. Из-за этого стоит выделить класс файлового объекта.

Поскольку чтение и запись файла по блокам одинакова для всех файловых систем, то можно реализовать методы `read-file`, `write-file`, `seek-file`, `tell-file`, а в

реализациях конкретных файловых систем вызывать методы родительского класса и менять метаданные файлов и каталогов.

Поле `blocks` в классе файла – список блоков файла.

Поле `position` в классе файла – пара из номера текущего блока и позиции в нём.

Если во время чтения позиция вышла за текущий блок, то проверяется есть ли у этого файла следующий блок, если блока нет, то вызывается ошибка, иначе номер следующего блока записывается в поле `position`, позиция в текущем блоке в поле `position` обнуляется.

При записи в файл может понадобиться добавление нового блока к файлу, для этого стоит добавить метод `new-block`, реализация которого будет в конкретной файловой системе.

Для остальных функций интерфейса виртуальной файловой выделяется класс файловой системы, реализация функций будет в конкретной файловой системе.

Для того чтобы раскрывать каталога в дереве файлов и каталогов добавляется метод `load-dir`, реализация которого будет в конкретной файловой системе.

Классы виртуальной файловой системы:

1. Класс абстрактной файловой системы. Этот класс отвечает за создание, удаление, переименование, открытие файлов и каталогов.
2. Класс абстрактного файла. Этот класс отвечает за чтение из файла и запись в него.

Каждая файловая система – два класса, являющиеся наследниками классов виртуальной файловой системы, которые реализуют функции интерфейса виртуальной файловой системы.

Модуль виртуальной файловой системы включает следующие классы:

1. `FileSystem` – класс абстрактной файловой системы. Методы класса `FileSystem`:

1.1. `fs-init` — загрузка файловой системы. Аргументы: `disk`(номер диска, с которого загружают), `start-sector`(номер сектора начала раздела), `sector-count`(количество секторов раздела). Возвращает `nil`.

1.2. `create-dir` — создать каталог. Аргументы: `dir`(каталог, в котором создать каталог), `name`(название каталога). Возвращает `nil`.

1.3. `remove-dir` — удалить каталог, если он пуст. Аргументы: `parent-dir`(каталог, из которого нужно удалить), `dir`(каталог, который нужно удалить). Возвращает `nil`.

1.4. `load-dir` — раскрыть и сохранить содержимое каталога. Аргументы: `dir`(каталог, чьё содержимое нужно загрузить). Возвращает `nil`.

1.5. `create-file` — создать и открыть файл. Аргументы: `dir`(каталог, в котором необходимо создать файл), `name`(название файла). Возвращает объект файла.

1.6. `remove-file` — удалить файл. Аргументы: `dir`(каталог, из которого нужно удалить), `entry`(файл, который нужно удалить). Возвращает `nil`.

1.7. `open-file` — открыть файл как поток для чтения и записи. Аргументы: `file`(файл, который нужно открыть). Возвращает объект файла.

1.8. `rename` — переименовать файл или каталог. Аргументы: `entry`(файл или каталог, который нужно переименовать), `new-name`(новое имя файла или каталога). Возвращает `nil`.

1.9. `free-space` — получить объём свободного места в текущем разделе диска. Аргументов нет. Возвращает размер в байтах.

1.10. `set-attr` — изменить атрибуты файла или каталога. Аргументы: `entry`(файл или каталог, чьи атрибуты необходимо поменять), `new-attributes`(новые атрибуты файла или каталога). Возвращает `nil`.

1.11. `fstat` — получить метаданные файла или каталога. Аргументы: `entry`(файл или каталог, чьи метаданные надо получить). Возвращает структуру `stat`. Структура `stat` состоит из:

1.11.1. `name` — название файла;

1.11.2. `size` — размер файла в байтах. Для каталогов равно 0;

1.11.3. `create-date` – дата создания файла, если нет в конкретной файловой системе, то равно `nil`;

1.11.4. `create-time` – время создания файла, если нет в конкретной файловой системе, то равно `nil`;

1.11.5. `modify-date` – дата последнего изменения файла, если нет в конкретной файловой системе, то равно `nil`;

1.11.6. `modify-time` – время последнего изменения файла, если нет в конкретной файловой системе, то равно `nil`;

1.11.7. `access-date` – дата последнего доступа файла, если нет в конкретной файловой системе, то равно `nil`;

1.11.8. `access-time` – время последнего изменения файла, если нет в конкретной файловой системе, то равно `nil`;

1.11.9. `isdir` – флаг, показывающий, является ли файл каталогом;

1.11.10. другие атрибуты, которые зависят от конкретной файловой системы.

1.12. `new-block` – добавить новый блок к файлу. Аргументы: `file-object`(объект файла, к которому необходимо добавить новый блок). Возвращает `nil`.

2. `File` – класс абстрактного файла. Поля класса `File`:

2.1. `name` – имя файла.

2.2. `size` – размер файла в байтах.

2.3. `position` – позиция в потоке чтения/записи файла (пара из номера текущего блока и позиции в нём).

2.4. `blocks` – список номеров блока файла.

Методы класса `File`:

2.1. `close-file` – закрыть файл. Аргументы: `file-object`(объект файла, который необходимо закрыть). Возвращает `nil`.

2.2. `read-file` – прочитать определённое количество байт из потока файла, и переместить позицию внутри потока на то же количество. Аргументы: `file-object`(объект файла, из которого читать байты), `size`(количество байт, которое необходимо прочитать). Возвращает массив прочитанных байт.

2.3. `write-file` — записать массив байт в поток файла, и переместить позицию внутри потока на то же количество. При необходимости добавить новый блок к файлу. Аргументы: `file-object`(объект файла, в который необходимо записать массив байт), `buffer`(массив байт, который необходимо записать). Возвращает количество байт, которые было записано.

2.4. `tell-file` — получить позицию внутри потока файла. Аргументы: `file-object`(объект файла, позицию в котором необходимо вернуть).

2.5. `seek-file` — переместить позицию внутри потока файла. Аргументы: `file-object`(объект файла, позицию в котором необходимо переместить), `offset`(насколько сместить позицию), `origin`(откуда смещать позицию). Возвращает значение новой позиции в потоке.

3.10 Модуль файловой системы FAT-32

Описание файловой системы FAT-32 приведено выше.

При загрузке файловой системы необходимо прочитать BPB, FSInfo, FAT.

Из BPB необходимо достать следующую информацию:

- размер блока в байтах;
- количество секторов зарезервированной области;
- размер таблицы FAT;
- количество секторов на одну таблицу FAT;
- номер активной таблицы FAT и режим копирования;
- номер первого блока корневого каталога;
- номер сектора активной структуры FSInfo.

Из FSInfo необходимо достать следующую информацию:

- количество свободных блоков (вычислить при необходимости);
- номер, с которого необходимо начать поиск свободного блока (вычислить при необходимости).

Вычисление количества свободных блоков и номера, с которого необходимо начать поиск свободного блока происходит в момент выделения нового блока.

Для удобства можно хранить всю таблицу FAT, её удобно хранить как хеш-объект с ключом - номер первого блока в цепочке и значением – список оставшихся блоков. При получении цепочки таблицы FAT – возвращать цепочку из хеш-объекта, а при записи – менять цепочку в хеш-объекте и записывать в таблицу FAT на диске.

При вызове функции открытия файла, создаётся экземпляр класса FAT32File. Список блоков файла сохраняется в поле blocks. Во время чтения или записи, позиция внутри текущего блока сохраняется в поле position, номер текущего блока сохраняется в поле position.

Модуль файловой системы FAT-32 включает следующие классы:

1. FAT32FileSystem – класс файловой системы FAT-32. Наследник класса FileSystem. Поля класса FAT32FileSystem:

- 1.1. fat-table – массив таблицы FAT.
- 1.2. fat-start-sector – номер сектора таблицы FAT.
- 1.3. fat-copy-start-sector – номер сектора копии таблицы FAT.
- 1.4. fat-sectors – количество секторов таблицы FAT.
- 1.5. free-blocks-count – количество свободных блоков.
- 1.6. free-block-num – номер, с которого необходимо начать поиск свободного блока.
- 1.7. root-dir-block – номер блока корневого каталога.

Методы класса FAT32FileSystem:

1.1. fs-init – загрузка файловой системы. Чтение BPB, FSInfo, таблицы FAT. Аргументы: disk(номер диска, с которого загружают), start-sector(номер сектора начала раздела), sector-count(количество секторов раздела). Возвращает nil.

1.2. create-dir – создать каталог. Создать записи длинного имени, создать запись в каталоге, выделить блок для каталога. Аргументы: dir(каталог, в котором создать каталог), name(название каталога). Возвращает nil.

1.3. remove-dir – удалить каталог, если он пуст. Аргументы: parent-dir(каталог, из которого нужно удалить), dir(каталог, который нужно удалить). Возвращает nil.

1.4. `load-dir` – раскрыть и сохранить содержимое каталога. Аргументы: `dir`(каталог, чьё содержимое нужно загрузить). Возвращает `nil`.

1.5. `create-file` – создать файл. Создать записи длинного имени, создать запись в каталоге, выделить блок для файла. Аргументы: `dir`(каталог, в котором необходимо создать файл), `name`(название файла). Возвращает `nil`.

1.6. `remove-file` – удалить файл. Заменить первые символы во всех записях длинного имени, а также в записи каталога, на 229 ($E5_{16}$). Аргументы: `dir`(каталог, из которого нужно удалить), `entry`(файл, который нужно удалить). Возвращает `nil`.

1.7. `open-file` – открыть файл как поток для чтения и записи. Аргументы: `file`(файл, который нужно открыть). Возвращает объект файла.

1.8. `rename` – переименовать файл или каталог. При необходимости удалить старые записи длинного имени, а также запись в каталоге, и создать новые записи длинного имени и новую запись в каталоге. Аргументы: `entry`(файл или каталог, который нужно переименовать), `new-name`(новое имя файла или каталога). Возвращает `nil`.

1.9. `free-space` – получить объём свободного места в текущем разделе диска. Аргументов нет. Возвращает размер в байтах.

1.10. `set-attr` – изменить атрибуты файла или каталога. Аргументы: `entry`(файл или каталог, чьи атрибуты необходимо поменять), `new-attributes`(новые атрибуты файла или каталога). Возвращает `nil`.

1.11. `fstat` – получить метаданные файла или каталога. Аргументы: `entry`(файл или каталог, чьи метаданные надо получить), `name`(имя файла, чьи метаданные надо получить). Возвращает структуру `stat`. Структура `stat` состоит из:

1.11.1. `name` – название файла.

1.11.2. `size` – размер файла в байтах. Для каталогов равно 0.

1.11.3. `create-date` – дата создания файла.

1.11.4. `create-time` – время создания файла.

1.11.5. `modify-date` – дата последнего изменения файла.

1.11.6. `modify-time` – время последнего изменения файла.

1.11.7. `access-date` – дата последнего доступа файла.

1.11.8. `access-date` – всегда имеет значение равное `nil`.

1.11.9. `isdir` – флаг, показывающий, является ли файл каталогом.

1.11.10. `create-time-ms` – сотые доли секунды создания файла или каталога (1 байт). Имеет значение от 0 до 199 (от 0 до 1.99 секунды с шагом 0.01 секунда).

1.11.11. `attribute` – атрибуты файла.

1.12. `new-block` – добавить новый блок к файлу. Аргументы: `file-object`(объект файла, к которому необходимо добавить новый блок). Возвращает `nil`.

2. `FAT32File` – класс файла файловой системы FAT-32. Наследник класса `File`. Поля класса `FAT32File`:

2.1. `name` – имя файла.

2.2. `size` – размер файла в байтах.

2.3. `position` – позиция в потоке чтения/записи файла (пара из номера текущего блока и позиции в нём).

2.4. `blocks` – список номеров блока файла.

2.5. `attributes` – флаги файла.

Методы класса `FAT32File`:

2.1. `close-file` – закрыть файл. Аргументы: `file-object`(объект файла, который необходимо закрыть). Возвращает `nil`.

2.2. `read-file` – прочитать определённое количество байт из потока файла, и переместить позицию внутри потока на то же количество. При необходимости обновить номер и содержимое текущего блока. Изменить дату последнего доступа. Аргументы: `file-object`(объект файла, из которого читать байты), `size`(количество байт, которое необходимо прочитать). Возвращает массив прочитанных байт.

2.3. `write-file` – записать массив байт в поток файла, и переместить позицию внутри потока на то же количество. При необходимости выделить новые блоки. При необходимости обновить номер и содержимое текущего блока. Изменить дату и время последнего изменения. Аргументы: `file-object`(объект

файла, в который необходимо записать массив байт), `buffer`(массив байт, который необходимо записать). Возвращает количество байт, которые было записано.

2.4. `tell-file` – получить позицию внутри потока файла. Аргументы: `file-object`(объект файла, позицию в котором необходимо вернуть).

2.5. `seek-file` – переместить позицию внутри потока файла. При необходимости обновить номер и содержимое текущего блока. Аргументы: `file-object`(объект файла, позицию в котором необходимо переместить), `offset`(насколько сместить позицию), `origin`(откуда смещать позицию). Возвращает значение новой позиции в потоке.

3.11 Модуль файловой системы ISO-9660 (CDFS)

Описание файловой системы ISO-9660 (CDFS) приведено выше.

При загрузке файловой системы необходимо прочитать Основной дескриптор тома, запись в каталоге для корневого каталога из основного дескриптора тома.

Из основного дескриптора тома необходимо достать следующую информацию:

- количество блоков раздела;
- размер одного блока;
- размер таблицы путей в байтах;
- номер сектора, в котором находится таблица путей в формате Little-Endian;
- запись в каталоге для корневого каталога.

Из записи в каталоге для корневого каталога необходимо достать следующую информацию:

- номер блока с содержимым (только часть с Little-Endian);
- размер содержимого в байтах (только часть с Little-Endian);
- дату и время записи;
- атрибуты корневого каталога.

При вызове функции открытия файла, создаётся экземпляр класса CDFSFile. Список блоков файла сохраняется в поле blocks. Во время чтения, позиция внутри текущего блока сохраняется в поле position, номер текущего блока сохраняется в поле position.

Модуль файловой системы ISO-9660 (CDFS) включает следующие классы:

1. CDFSFileSystem – класс файловой системы ISO-9660 (CDFS). Наследник класса FileSystem. Поля класса CDFSFileSystem:

1.1. root-entry – запись в каталоге корневого каталога (номер блока с содержимым, размер содержимого, дата и время записи, атрибуты).

Методы класса CDFSFileSystem:

1.1. fs-init – загрузка файловой системы. Чтение основного дескриптора тома, записи в каталоге для корневого каталога из основного дескриптора тома. Аргументы: disk(номер диска, с которого загружают), start-sector(номер сектора начала раздела), sector-count(количество секторов раздела). Возвращает nil.

1.2. create-dir – файловая система ISO-9660 (CDFS) только для чтения, поэтому запись невозможна. Аргументов нет. Вызывает ошибку, что файловая система ISO-9660 (CDFS) только для чтения.

1.3. remove-dir – файловая система ISO-9660 (CDFS) только для чтения, поэтому запись невозможна. Аргументов нет. Вызывает ошибку, что файловая система ISO-9660 (CDFS) только для чтения.

1.4. load-dir – раскрыть и сохранить содержимое каталога. Аргументы: dir(каталог, чьё содержимое нужно загрузить). Возвращает nil.

1.5. create-file – файловая система ISO-9660 (CDFS) только для чтения, поэтому запись невозможна. Аргументов нет. Вызывает ошибку, что файловая система ISO-9660 (CDFS) только для чтения.

1.6. remove-file – файловая система ISO-9660 (CDFS) только для чтения, поэтому запись невозможна. Аргументов нет. Вызывает ошибку, что файловая система ISO-9660 (CDFS) только для чтения.

1.7. `open-file` – открыть файл как поток для чтения и записи. Аргументы: `file`(файл, который нужно открыть). Возвращает объект файла.

1.8. `rename` – файловая система ISO-9660 (CDFS) только для чтения, поэтому запись невозможна. Аргументов нет. Вызывает ошибку, что файловая система ISO-9660 (CDFS) только для чтения.

1.9. `free-space` – файловая система ISO-9660 (CDFS) только для чтения, поэтому запись невозможна. Аргументов нет. Вызывает ошибку, что файловая система ISO-9660 (CDFS) только для чтения.

1.10. `set-attr` – файловая система ISO-9660 (CDFS) только для чтения, поэтому запись невозможна. Аргументов нет. Вызывает ошибку, что файловая система ISO-9660 (CDFS) только для чтения.

1.11. `fstat` – получить метаданные файла или каталога. Аргументы: `dir`(каталог с файлом, чьи метаданные надо получить), `name`(имя файла, чьи метаданные надо получить). Возвращает структуру `stat`. Структура `stat` состоит из:

1.11.1. `name` – название файла.

1.11.2. `size` – размер файла в байтах. Для каталогов равно 0.

1.11.3. `create-date` – дата создания файла.

1.11.4. `create-time` – время создания файла.

1.11.5. `modify-date` – всегда имеет значение равное `nil`.

1.11.6. `modify-time` – всегда имеет значение равное `nil`.

1.11.7. `access-date` – всегда имеет значение равное `nil`.

1.11.8. `access-time` – всегда имеет значение равное `nil`.

1.11.9. `isdir` – флаг, показывающий, является ли файл каталогом.

1.11.10. `flags` – атрибуты файла.

1.12. `new-block` – добавить новый блок к файлу. Аргументы: `file-object`(объект файла, к которому необходимо добавить новый блок). Вызывает ошибку, что файловая система ISO-9660 (CDFS) только для чтения.

2. `CDFSFile` – класс файла файловой системы ISO-9660 (CDFS). Наследник класса `File`. Поля класса `CDFSFile`:

- 2.1. name – имя файла.
- 2.2. size – размер файла в байтах.
- 2.3. position – позиция в потоке чтения/записи файла (пара из номера текущего блока и позиции в нём).
- 2.4. blocks – список номеров блока файла.

Методы класса CDFSFile:

2.1. close-file – закрыть файл. Аргументы: file-object(объект файла, который необходимо закрыть). Возвращает nil.

2.2. read-file – прочитать определённое количество байт из потока файла, и переместить позицию внутри потока на то же количество. При необходимости обновить номер и содержимое текущего блока. Изменить дату последнего доступа. Аргументы: file-object(объект файла, из которого читать байты), size(количество байт, которое необходимо прочитать). Возвращает массив прочитанных байт.

2.3. write-file – файловая система ISO-9660 (CDFS) только для чтения, поэтому запись невозможна. Аргументов нет. Вызывает ошибку, что файловая система ISO-9660 (CDFS) только для чтения.

2.4. tell-file – получить позицию внутри потока файла. Аргументы: file-object(объект файла, позицию в котором необходимо вернуть).

2.5. seek-file – переместить позицию внутри потока файла. При необходимости обновить номер и содержимое текущего блока. Аргументы: file-object(объект файла, позицию в котором необходимо переместить), offset(насколько сместить позицию), origin(откуда смещать позицию). Возвращает значение новой позиции в потоке.

3.12 Модуль работы с блоками диска

Блок – один или несколько секторов подряд.

Из-за того, что размер блока у разных файловых систем может быть разным (он определяется при загрузке файловой системы), необходима глобальная переменная block-sectors, которая будет хранить количество секторов на один блок.

Также из-за того, что у файловых систем разные номера первого блока области данных, необходима глобальная переменная `block-offset`, которая будет хранить смещение в секторах от начала раздела для корректной нумерации блоков.

Из-за того, что номер первого блока области данных у разных файловых систем имеет разное значение, необходима глобальная переменная `blocks-start-num`, которая будет хранить минимальный номер блока области данных, чтобы при возникновении ошибок (в качестве номера блока для чтения или записи попадает номер, который указывает на блок не в области данных) не перезаписать часть структуры файловых систем.

Модуль работы с блоками диска включает следующие переменные:

1. `disk-num` – номер диска, с которого читают блоки.
2. `block-offset` – смещение в секторах для отсчёта блоков.
3. `block-sectors` – количество секторов на один блок.
4. `blocks-start-num` – минимальный номер блока для чтения или записи.

Модуль работы с блоками диска включает следующие функции:

1. `set-block-disk` – установить диск, с которого читать блоки. Аргументы: `disk`(номер диска, с которого читать блоки). Возвращает `nil`.
2. `set-block-size` – установить размер блока диска. Аргументы: `size`(размер блока в секторах). Возвращает `nil`.
3. `set-block-offset` – установить смещение в секторах для отсчёта блоков диска. Аргументы: `offset`(смещение в секторах). Возвращает `nil`.
4. `set-blocks-start-num` – установить минимальный номер блока для чтения или записи. Аргументы: `blocks-start-num`(минимальный номер блока для чтения или записи). Возвращает `nil`.
5. `block-read` – прочитать блок диска. Аргументы: `num`(номер блока, который читают). Возвращает массив прочитанных байт.
6. `block-write` – записать в блок диска. Аргументы: `num`(номер блока, в который записывают), `buf`(массив байт). Возвращает `nil`.

7. `get-blocks-pos` – получить номер блока и смещение по позиции в списке блоков. Аргументы: `blocks`(список блоков, в котором ищут позицию), `pos`(смещение в байтах). Возвращает пару из номера блока и позиции в нём.

8. `get-offset-from-pos` – получить смещение в байтах из позиции. Аргументы: `pos`(индекс блока и смещение в блоке). Возвращает смещение в байтах.

3.13 Модуль драйвера АТА

Интерфейс АТА используется для работы с секторами диска.

Сектор диска – минимальная единица хранения информации на диске.

Модуль драйвера АТА включает следующие функции:

1. `ata-get-status-reg` – получение значения бита регистра статуса. Аргументы: `bit`(номер бита регистра статуса). Возвращает значение бита регистра статуса.

2. `ata-set-dev` – установить номер устройства. Аргументы: `dev`(номер устройства). Возвращает `nil`.

3. `ata-set-lba` – установить номер сектора LBA. Аргументы: `sec`(номер первого сектора), `num`(количество секторов). Возвращает `nil`.

4. `ata-check-error` – проверить на ошибку. Аргументов нет. Возвращает `t`, если есть ошибка, иначе `nil`.

5. `ata-set-command` – установить команду контроллера. Аргументы: `cmd`(команда контроллера). Возвращает `nil`.

6. `ata-read` – прочитать данные контроллера. Аргументы: `size`(количество байт). Возвращает массив прочитанных байт.

7. `ata-write` – записать массив в контроллер. Аргументы: `arr`(массив байт). Возвращает `nil`.

8. `ata-identify` – прочитать служебную информацию. Аргументов нет. Возвращает массив со служебной информацией: число головок, цилиндров, секторов, серийный номер и т.д.

9. `ata-read-sectors` – прочитать с секторов диска. Аргументы: `dev`(номер устройства), `start`(номер первого сектора), `num`(количество секторов). Возвращает массив прочитанных байт.

10. `ata-write-sectors` – записать массив байт на сектора диска. Аргументы: `dev`(номер устройства), `start`(номер первого сектора), `num`(количество секторов), `arr`(массив байт). Возвращает `nil`.

4 Рабочий проект

Можно выделить несколько модулей, использованных при разработке виртуальной файловой системы: командный интерпретатор, интерфейс виртуальной файловой системы, главная загрузочная запись, файловая система FAT-32, файловая система ISO-9660 (CDFS), виртуальная файловая система, работа с блоками диска, драйвер ATA.

4.1 Модуль «shell.lsp»

Функции модуля интерфейса виртуальной файловой системы представлены в таблице 4.1.

Таблица 4.1 – Описание функций, используемых в модуле командного интерпретатора

Название функции	Аргументы	Описание функции
1	2	3
key-down-handler	scan	Обработчик нажатия клавиш с клавиатуры, где scan – скан код нажатой клавиши.
process-command	com	Обработка введённой команды, где com – список введённых символов.
print-start	Аргументов нет	Вывод на экран строки активности командного интерпретатора.
parse-commands	Аргументов нет	Парсер всех возможных команд командного интерпретатора. Выполняет соответствующую команду.
parse-command	com fun	Парсер определённой команды, где com – строка названия команды, fun – функция, которую выполняет команда.
parse-elem-str	str	Предикат проверки корректности названия введённой команды, где str – строка названия команды.
get-str	Аргументов нет	Получить строку названия команды.
is-com-separator	c	Предикат проверки символа c на принадлежность к разделителям.

Продолжение таблицы 4.1

1	2	3
cat-command-action	args	Выполнение команды вывода содержимого файла на экран с аргументами args(путь).
copy-command-action	args	Выполнение команды копирования файла с аргументами args(путь-откуда путь-куда).
append-command-action	args	Выполнение команды добавление текста в конец файла с аргументами args(путь текст).
chattr-command-action	args	Выполнение команды изменения атрибутов файла или каталога с аргументами args(путь атрибуты).

4.2 Модуль «vfs.lsp»

Функции модуля интерфейса виртуальной файловой системы представлены в таблице 4.2.

Таблица 4.2 – Описание функций, используемых в модуле интерфейса виртуальной файловой системы

Название функции	Аргументы	Описание функции
1	2	3
cur-dir	Аргументов нет	Получение пути текущего каталога.
list-dir	path	Получение содержимого каталога по пути path.
create-dir	path	Создание каталога по пути path где последний элемент пути это название нового каталога.
remove-dir	path	Удаление каталога по пути path если он пуст.
change-dir	path	Перемещение в каталог по пути path.
create-file	path	Создание файла по пути path где последний элемент пути это название нового файла.
remove-file	path	Удаление файла по пути path.

Продолжение таблицы 4.2

1	2	3
open-file	path	Открытие файла как поток для чтения и записи по пути path и вернуть файловый объект.
is-directory	path	Проверка файла по пути path на каталог.
rename	path name	Переименовывание файла или каталога по пути path на name.
free-space	Аргументов нет	Получение объёма свободного места в разделе.
fstat	path	Получение метаданных файла или каталога.
set-attr	path new-attributes	Изменение атрибутов файла или каталога по пути path на new-attributes.
find-entry	dir name	Поиск файла или каталога с названием name в каталоге dir.
parse-path	str-path	Преобразование строки пути str-path в список пути.
parse-up-dir	path-list	Преобразовать список пути path-list убрав все "..".
load-path	path	Проверяет и при необходимости загружает путь path. Возвращает файловый объект файла или каталога или вызывает ошибку если пути нет.
get-parent-dir	path	Получение файлового объекта родительского каталога файла или каталога по строке пути path.

4.3 Модуль «mbr.lsp»

Функции модуля главной загрузочной записи представлены в таблице 4.3.

Таблица 4.3 – Описание функций, используемых в модуле главной загрузочной записи

Название функции	Аргументы	Описание функции
1	2	3
parse-partition-rec	Аргументов нет	Парсер записи таблицы раздеров в MBR.
load-partition	disk-num part-num	Загрузить файловую систему из раздела part-num с диска disk-num.

4.4 Модули «fat32-class.lsp», «fat32-fat.lsp», «fat32-entry.lsp», «fat32-fs.lsp», «fat32-file.lsp»

Функции модуля файловой системы FAT-32 представлены в таблице 4.4.

Таблица 4.4 – Описание функций, используемых в модуле файловой системы FAT-32

Название функции	Аргументы	Описание функции
1	2	3
parse-bios-parameter-block	Аргументов нет	Парсер блока параметров BIOS.
parse-fsinfo	Аргументов нет	Парсер структуры FSInfo.
update-fsinfo	free-count free-num	Обновить значения структуры fsinfo на диске на free-count и free-num.
fat-elem-pos	idx	Получить пару из смещение сектора + смещение в секторе для элемента с индексом idx таблицы FAT.
fat-read-sector	fat-table sector-num	Прочитать сектор sector-num таблицы FAT, записав содержимое сектора таблицы в массив fat-table.

Продолжение таблицы 4.4

1	2	3
fat-check-for-read	fat-table fat-read-sectors elem-num	Проверить прочитан ли сектор таблицы FAT, в котором находится элемент elem-num через массив прочитанных секторов fat-read-sectors. Если нет то прочитать сектор таблицы FAT в fat-table.
get-fat-chain	block-num	Получить цепочку блоков из таблицы FAT, начиная с блока block-num.
get-free-block	fat-table fat-read-sectors free-block-num	Найти первый свободный блок в таблице FAT fat-table, где массив прочитанных секторов таблицы FAT fat-read-sectors, начиная поиск с free-block-num.
update-fat-elem	idx val	Установить элемент с индексом idx в таблице FAT на диске на значение val при необходимости обновить во всех копиях таблицы FAT.
append-fat-chain	first-block new-idx	Добавить элемент new-idx в цепочку блоков с первым блоком first-block.
fat32-is-special-name	name	Проверка на специальное имя каталога.
fat32-get-date-time	date-bytes time-bytes creation-time-ms	Получить список даты и времени из массивов байт date-bytes, time-bytes и creation-time-ms.
fat32-generate-date-time	date-time	Получить массив байт для даты и времени для записи в каталоге из списка даты и времени date-time.
fat32-get-attributes	attribute-byte	Получить список атрибутов из записи в каталоге из байта attribute-byte.
fat32-generate-attribute-byte	attributes	Получить байт атрибутов для записи в каталоге из списка атрибутов attributes.
parse-lfn-entry	Аргументов нет	Парсер структуры записи длинного имени.
parse-fat32-dir-entry	Аргументов нет	Парсер структур записи в каталоге.

Продолжение таблицы 4.4

1	2	3
fat32-calc-lfn-count	name	Посчитать количество записей длинного имени для имени name.
fat32-generate-checksum	short-name	Посчитать контрольную сумму короткого имени для записи длинного имени.
fat32-create-lfn-entry	name-part-str lfn-num checksum	Создать одну запись длинного имени для строки name-part-str, с номером lfn-num и контрольной суммой checksum.
fat32-create-lfn-entries	entry	Создать записи длинного имени для записи в каталоге entry.
fat32-create-dir-entry	entry	Создать массив байт записи в каталоге entry.
fat32-create-lfn-dir-entries	entry	Создать массив байт записей длинного имени и записи в каталоге entry.
fat32-find-entry-pos	entry	Найти позицию (смещение в списке блоков . смещение в блоке) для записи в каталоге entry в каталоге dir.
fat32-get-free-entry-pos	dir-first-block entries-count	Найти позицию (смещение в списке блоков . смещение в блоке) в каталоге, где первый блок dir-first-block, где сможет поместиться количество записей entries-count. При необходимости выделяется новый блок.
fat32-mark-for-delete	entry	Пометить записей длинного имени и записи в каталоге entry на удаление.
fat32-add-entry	entry	Добавление записи entry на диск.
fat32-update-entry	entry changes	Обновить запись в каталоге entry выполнив изменения changes. Если количество записей длинного имени для новой записи отличается, то записи длинного имени и запись в каталоге будет пересоздана.
is-short-name-sym	symbol	Проверка на символ короткой строки.

Продолжение таблицы 4.4

1	2	3
get-trail-num	dir-entries short-name	Получить номер числового хвоста для короткого имени short-name.
fat32-generate-short-name	dir long-name	Сгенерировать короткое имя из длинного имени long-name в каталоге dir.
fat32-sort-attributes	attributes is-dir	Отсортировать список атрибутов attribute с проверкой атрибута DIRECTORY в зависимости от того является ли файл каталогом (is-dir).
fat32-create-special-entries	dir	Создать особые файлы указывающие на текущий каталог и на родительский каталог.
fat32-is-name-duplicate	dir name	Проверить есть ли в каталоге dir файл с названием name.

4.4.1 Класс «FAT32FileSystem»

«FAT32FileSystem» – класс файловой системы FAT-32, методы которого являются функциями интерфейса виртуальной файловой системы.

Методы класса «FAT32FileSystem» представлены в таблице 4.5

Таблица 4.5 – Описание методов класса «FAT32FileSystem»

Название метода	Аргументы	Описание метода
1	2	3
fs-init	(fat32fs FAT32FileSystem) disk start-sector sector-count	Загрузка файловой системы FAT32 с диска disk, начиная с сектора start-sector, с количеством секторов sector-count. Чтение блока параметров BIOS, структуры FSInfo, таблицы FAT.
load-dir	(fat32fs FAT32FileSystem) dir	Раскрыть и сохранить содержимое каталога dir.

Продолжение таблицы 4.5

1	2	3
fstat*	(fat32fs FAT32FileSystem) file-hash	Получение метаданных файлового объекта file-hash.
open-file*	(fat32fs FAT32FileSystem) file-hash	Открыть файл file-hash как поток для чтения и записи. Изменить время последнего доступа.
new-block	(fat32fs FAT32FileSystem) file-object	Выделить новый блок для файла file-object.
rename*	(fat32fs FAT32FileSystem) entry new-name	Переименовать файл или каталог entry на new-name.
remove-file*	(fat32fs FAT32FileSystem) dir entry	Пометить файл entry на удаление.
remove-dir*	(fat32fs FAT32FileSystem) parent-dir dir	Пометить каталог entry на удаление если он пустой.
free-space*	(fat32fs FAT32FileSystem)	Получить объём свободного места в текущем разделе в байтах.
create-file*	(fat32fs FAT32FileSystem) dir name	Создать файл с названием name в каталоге dir.
create-dir*	(fat32fs FAT32FileSystem) dir name	Создать каталог с названием name в каталоге dir.
set-attr*	(fat32fs FAT32FileSystem) entry new-attributes	Изменить атрибуты файла entry на new-attributes.

4.4.2 Класс «FAT32File»

«FAT32File» – класс файла файловой системы FAT-32, методы которого являются функциями интерфейса виртуальной файловой системы.

Методы класса «FAT32File» представлены в таблице 4.6

Таблица 4.6 – Описание методов класса «FAT32File»

Название метода	Аргументы	Описание метода
1	2	3
close-file	(self FAT32File)	Заккрытие файла.
tell-file	(self FAT32File)	Получение позиции в файле.
seek-file	(self FAT32File) offset origin	Сместить позицию в файле на offset, начиная с origin(SET (начало) CUR (текущее) END (конец)).
read-file	(self FAT32File) size	Прочитать size байт из файла и сместить позицию на тоже число. Изменить время последнего доступа.
write-file	(self FAT32File) buf	Записать в файл массив байт buf, сместив позицию на соответствующее число. Изменить время последнего доступа, время последнего изменения. При необходимости выделить новый блок.

4.5 Модули «iso9660-fs.lsp», «iso9660-file.lsp»

Функции модуля файловой системы ISO-9660 (CDFS) представлены в таблице 4.7.

Таблица 4.7 – Описание функций, используемых в модуле файловой системы ISO-9660 (CDFS)

Название функции	Аргументы	Описание функции
1	2	3
parse-boot-record	Аргументов нет	Парсер оставшейся части структуры дескриптора типа Boot Record.

Продолжение таблицы 4.7

1	2	3
parse-primary-desc	Аргументов нет	Парсер оставшейся части структуры дескриптора типа Primary Volume Descriptor.
parse-set-terminator-desc	Аргументов нет	Парсер оставшейся части структуры дескриптора типа Volume Descriptor Set Terminator.
parse-descriptor-data	Аргументов нет	Возврат парсера оставшейся части структуры дескриптора типа desc-type.
parse-date-time-entry	Аргументов нет	Парсер структуры даты и времени в записи в каталоге.
cdfs-get-date-time-entry	date-time-bytes	Получить список даты и времени из записи в каталоге из массива байт date-time-bytes.
cdfs-get-attributes-entry	attr-byte	Получить список атрибутов из записи в каталоге из байта attr-byte.
parse-cdfs-dir-entry	Аргументов нет	Парсер структуры записи в каталоге.
parse-descriptor	Аргументов нет	Парсер структура основы дескриптора.

4.5.1 Класс «CDFSFileSystem»

«CDFSFileSystem» – класс файловой системы ISO-9660 (CDFS), методы которого являются функциями интерфейса виртуальной файловой системы.

Методы класса «CDFSFileSystem» представлены в таблице 4.8

Таблица 4.8 – Описание методов класса «CDFSFileSystem»

Название метода	Аргументы	Описание метода
1	2	3
fs-init	(self CDFSFileSystem) disk start-sector sector-count	Загрузка файловой системы ISO9660 с диска disk, начиная с сектора start-sector, с количеством секторов sector-count. Чтение основного дескриптора тома, записи в каталоге для корневого каталога из основного дескриптора тома.
load-dir	(self CDFSFileSystem) dir	Раскрыть и сохранить содержимое каталога dir.
fstat*	(self CDFSFileSystem) file-hash	Получение метаданных файлового объекта file-hash.
open-file*	(self CDFSFileSystem) file-hash	Открыть файл как поток для чтения.
new-block	(self CDFSFileSystem) file-object	Вызов ошибки при попытке выделения нового блока в файловой системе CDFS (ISO9660).
rename*	(self CDFSFileSystem) entry new-name	Вызов ошибки при попытке переименовании файла или каталога в файловой системе CDFS (ISO9660).
remove-file*	(self CDFSFileSystem) dir entry	Вызов ошибки при попытке переименовании файла в файловой системе CDFS (ISO9660).
remove-dir*	(self CDFSFileSystem) parent-dir dir	Вызов ошибки при попытке переименовании каталога в файловой системе CDFS (ISO9660).
free-space*	(self CDFSFileSystem)	Вызов ошибки при попытке получения свободного места в файловой системе CDFS (ISO9660).

Продолжение таблицы 4.8

1	2	3
create-file*	(self CDFSFileSystem) dir name	Вызов ошибки при попытке создании файла в файловой системе CDFS (ISO9660).
create-dir*	(self CDFSFileSystem) dir name	Вызов ошибки при попытке создании каталога в файловой системе CDFS (ISO9660).
set-attr*	(self CDFSFileSystem) entry new- attributes	Вызов ошибки при попытке изменения атрибутов файла или каталога в файловой системе CDFS (ISO9660).

4.5.2 Класс «CDFSFile»

«CDFSFile» – класс файла файловой системы ISO-9660 (CDFS), методы которого являются функциями интерфейса виртуальной файловой системы.

Методы класса «CDFSFile» представлены в таблице 4.9

Таблица 4.9 – Описание методов класса «CDFSFile»

Название метода	Аргументы	Описание метода
1	2	3
close-file	(self CDFSFile)	Закрытие файла.
tell-file	(self CDFSFile)	Получение позиции в файле.
seek-file	(self CDFSFile) offset origin	Сместить позицию в файле на offset, начиная с origin (SET (начало) CUR (текущее) END (конец)).
read-file	(self CDFSFile) size	Прочитать size байт из файла и сместить позицию на тоже число.
write-file	(self CDFSFile) buf	Вызов ошибки при попытке записи в файл в файловой системе ISO-9660 (CDFS).

4.6 Модуль «fs.lsp»

4.6.1 Класс «FileSystem»

«FileSystem» – класс абстрактной файловой системы, методы которого являются функциями интерфейса виртуальной файловой системы. Реализации конкретных файловых систем наследуются от этого класса и реализуют его методы.

Методы класса «FileSystem» представлены в таблице 4.10

Таблица 4.10 – Описание методов класса «FileSystem»

Название метода	Аргументы	Описание метода
1	2	3
fs-init	(self FileSystem) disk start-sector sector-count	пустой метод, реализация которого находится в конкретной файловой системе.
load-dir	(self FileSystem) dir	пустой метод, реализация которого находится в конкретной файловой системе.
fstat*	(self FileSystem) file-hash	пустой метод, реализация которого находится в конкретной файловой системе.
open-file*	(self FileSystem) file-hash	пустой метод, реализация которого находится в конкретной файловой системе.
new-block	(self FileSystem) file-object	пустой метод, реализация которого находится в конкретной файловой системе.
rename*	(self FileSystem) entry new-name	пустой метод, реализация которого находится в конкретной файловой системе.
remove-file*	(self FileSystem) dir entry	пустой метод, реализация которого находится в конкретной файловой системе.
remove-dir*	(self FileSystem) parent-dir dir	пустой метод, реализация которого находится в конкретной файловой системе.

Продолжение таблицы 4.10

1	2	3
free-space*	(self FileSystem)	пустой метод, реализация которого находится в конкретной файловой системе.
create-file*	(self FileSystem) dir name	пустой метод, реализация которого находится в конкретной файловой системе.
create-dir*	(self FileSystem) dir name	пустой метод, реализация которого находится в конкретной файловой системе.
set-attr*	(self FileSystem) entry new- attributes	пустой метод, реализация которого находится в конкретной файловой системе.

4.6.2 Класс «File»

«File» – класс абстрактного файла, методы которого являются функциями интерфейса виртуальной файловой системы. Реализации конкретных файлов наследуются от этого класса и в методах реализуют изменение метаданных файла с вызовом метода этого класса.

Методы класса «File» представлены в таблице 4.11

Таблица 4.11 – Описание методов класса «File»

Название метода	Аргументы	Описание метода
1	2	3
close-file	(self File)	Закрытие файла.
tell-file	(self File)	Получение позиции в файле.
seek-file	(self File) offset origin	Сместить позицию в файле на offset, начиная с origin (SET (начало) CUR (текущее) END (конец)).
read-file	(self File) size	Прочитать size байт из файла и сместить позицию на тоже число.
write-file	(self File) buf	Записать в файл массив байт buf, сместив позицию на соответствующее число.

4.7 Модуль «block.lsp»

Функции модуля работы с блоками диска представлены в таблице 4.12.

Таблица 4.12 – Описание функций, используемых в модуле работы с блоками

Название функции	Аргументы	Описание функции
1	2	3
set-block-disk	disk	Установить диск disk, с которого читать блоки.
set-block-size	size	Установить размер блока диска size в секторах.
set-block-offset	offset	Установить смещение в секторах для отсчета блоков диска.
set-blocks-start-num	start-num	Установить значение минимального номера первого блока.
block-read	num	Прочитать блок с номером num.
block-write	num buf	Записать буфер buf в блок с номером num.
get-blocks-pos	blocks offset	Получить индекс блока и смещение по смещению offset в списке блоков blocks.
get-offset-from-pos	pos	Получить смещение в байтах из позиции (индекс блока . смещение в блоке).

4.8 Модуль «ata.lsp»

Функции модуля драйвера АТА представлены в таблице 4.13.

Таблица 4.13 – Описание функций, используемых в модуле главной загрузочной записи

Название функции	Аргументы	Описание функции
1	2	3
ata-get-status-reg	bit	Получение значения бита bit регистра статуса.

Продолжение таблицы 4.13

1	2	3
mk/ata-wait-status	name bit val	Генерация функций ожидания бита регистра статуса. name - имя, bit - номер бита статуса, val - значение для окончания ожидания.
ata-set-dev	dev	Установка номера устройства.
ata-set-lba	sec num	Установить номер сектора в режиме LBA.
ata-check-error	Аргументов нет	Проверка ошибки.
ata-set-command	cmd	Установить команду контроллера.
ata-read	size	Чтение данных контроллера, size байт.
ata-write	arr	Запись массива arr в контроллер.
ata-identify	Аргументов нет	Чтение служебной информации: число головок, цилиндров, секторов, серийный номер, ...
ata-read-sectors	dev start num	Читает сектора жесткого диска. dev - устройство: 0 primary master, 1 - slave. start - начальный сектор, num - число секторов. Возвращает массив данных.
ata-write-sectors	dev start num arr	Пишет сектора жесткого диска. dev - устройство: 0 primary master, 1 - slave. start - начальный сектор, num - число секторов, arr - массив байт данных.

4.9 Модульное и интеграционное тестирование разработанной виртуальной файловой системы

Результаты интеграционного тестирования модуля «vfs.lsp» представлены на рисунках 4.1 и 4.2

```

1  "join test"
2  "parse path test"
3  "load path test"
4  "get parent dir test"
5  "list dir test"
6  "create dir test"
7  "remove dir test"
8  "change dir test"
9  "create file test"
10 "remove file test"
11 "open file test"
12 "is directory test"
13 "rename test"
14 "free space test"
15 "fstat test"
16 "work with file test"
17 "set attributes test"
18 "Успешно завершено тестов: " 26 / 26
19 "Все тесты завершены успешно"

```

Рисунок 4.1 – Результат интеграционного тестирования функций модуля «vfs.lsp»

```

1  "MBR INIT"
2  "FAT32 PARTITION INIT"
3  "CDFS PARTITION INIT"
4  "FAT32 load partition"
5  "Create first file named 'first-file.txt'"
6  "write to file"
7  "read the content of file:" "wrote then read the content of first file"
8  "Create dir named 'firstdir'"
9  "change-dir to new dir"
10 "create second file named 'second file.bin'"
11 "write to file"
12 "read the content of file:" "wrote then read the content of second
    created file"
13 "CDFS load partition"
14 "open first file"
15 "content of first file"
16 "change-dir to second dir"
17 "open second file"
18 "first part of second file on first block"
19 "second part of second file on second block"

```

Рисунок 4.2 – Результат интеграционного тестирования модуля «vfs.lsp»

Результат интеграционного тестирования модуля «mbr.lsp» представлен на рисунке 4.3

```
1 "parse partition rec test"  
2 "load partition test"  
3 "Успешно завершено тестов: " 2 / 2  
4 "Все тесты завершены успешно"
```

Рисунок 4.3 – Результат интеграционного тестирования функций модуля «mbr.lsp»

Результаты интеграционного тестирования модулей «fat32-class.lsp», «fat32-fat.lsp», «fat32-entry.lsp», «fat32-fs.lsp», «fat32-file.lsp» представлены на рисунках 4.4, 4.5 и 4.6

```

1 "parse bios parameter block test"
2 "parse fsinfo test"
3 "get creation date-time test"
4 "get modify date-time test"
5 "get access date-time test"
6 "get attributes test 1"
7 "get attributes test 2"
8 "fat elem pos test"
9 "get fat chain test"
10 "parse lfn test 1"
11 "parse lfn test 2"
12 "parse dir entry test"
13 "fs-init test"
14 "load dir test 1"
15 "load dir test 2"
16 "fstat test"
17 "generate creation date-time bytes test"
18 "generate modify date-time bytes test"
19 "generate access date-time bytes test"
20 "generate attribute byte test 1"
21 "generate attribute byte test 2"
22 "calculate lfn count test 1"
23 "calculate lfn count test 2"
24 "calculate lfn count test 3"
25 "generate checksum test"
26 "generate checksum test"
27 "create lfn entry test 1"
28 "create lfn entry test 2"
29 "create lfn entries test"
30 "create dir entry test"
31 "create both lfn and dir entry test"
32 "find entry pos test"
33 "update entry test"
34 "get free entry pos test"
35 "mark entries for delete test"
36 "get free block test"
37 "update fat elem test1"
38 "update fat elem test2"
39 "append fat chain test"
40 "update entry test1"
41 "update entry test2"
42 "new block test"
43 "generate short name test 1"
44 "generate short name test 2"
45 "fat32 add entry"
46 "rename test"
47 "remove file test"
48 "remove dir test"
49 "free space test"

```

Рисунок 4.4 – Результат интеграционного тестирования функций модулей «fat32-class.lsp», «fat32-fat.lsp», «fat32-entry.lsp», «fat32-fs.lsp», «fat32-file.lsp»

```

1  "create file test"
2  "create dir test"
3  "open file test"
4  "read file test"
5  "write file test"
6  "sort attributes test"
7  "set attributes test"
8  "Успешно завершено тестов: " 56 / 56
9  "Все тесты завершены успешно"

```

Рисунок 4.5 – Продолжение результатов интеграционного тестирования функций модулей «fat32-class.lsp», «fat32-fat.lsp», «fat32-entry.lsp», «fat32-fs.lsp», «fat32-file.lsp»

```

1  "DISK INIT START"
2  "DISK INIT END"
3  "FS-INIT START"
4  "FS-INIT END"
5  "EMPTY ROOT DIR LOAD"
6  "CREATE FIRST FILE AND FIRST DIR"
7  "OPEN FIRST FILE"
8  "FIRST FILE WRITE"
9  "FIRST FILE READ"
10 "first file content"
11 "EMPTY FIRST DIR LOAD"
12 "CREATE SECOND FILE IN FIRST DIR"
13 "OPEN SECOND FILE"
14 "SECOND FILE WRITE"
15 "SECOND FILE READ"
16 "second file content"
17 "CREATE SECOND DIR"
18 "EMPTY SECOND DIR LOAD"
19 "CREATE THIRD FILE IN SECOND DIR"
20 "OPEN THIRD FILE"
21 "THIRD FILE WRITE"
22 "THIRD FILE READ"
23 "third file content"

```

Рисунок 4.6 – Результат интеграционного тестирования модулей «fat32-class.lsp», «fat32-fat.lsp», «fat32-entry.lsp», «fat32-fs.lsp», «fat32-file.lsp»

Результаты интеграционного тестирования модулей «iso9660-fs.lsp», «iso9660-file.lsp» представлены на рисунках 4.7 и 4.8


```

1 "get date time from entry test"
2 "get attributes from entry test 1"
3 "get attributes from entry test 2"
4 "get attributes from entry test 3"
5 "parse root dir entry test"
6 "parse dir entry test 1"
7 "parse dir entry test 2"
8 "parse dir entry test 3"
9 "parse boot record descriptor test"
10 "parse descriptor set terminator test"
11 "parse primary volume descriptor test"
12 "fs init test"
13 "load dir test"
14 "fstat test"
15 "open file test"
16 "read file test"
17 "Успешно завершено тестов: " 16 / 16
18 "Все тесты завершены успешно"

```

Рисунок 4.7 – Результат интеграционного тестирования функций модулей «iso9660-fs.lsp», «iso9660-file.lsp»

```

1 DISK-MAKE-START
2 DISK-MAKE-END
3 FS-INIT-START
4 FS-INIT-END
5 LOAD-ROOT-DIR-START
6 LOAD-ROOT-DIR-END
7 "content of first file"
8 LOAD-DIR-START
9 LOAD-DIR-END
10 "first part of second file on first block"
11 "second part of second file on second block"

```

Рисунок 4.8 – Результат интеграционного тестирования модулей «iso9660-fs.lsp», «iso9660-file.lsp»

Результат интеграционного тестирования модуля «fs.lsp» представлен на рисунке 4.9

```

1  "close file test"
2  "tell file test"
3  "seek file test1"
4  "seek file test2"
5  "seek file test2.2"
6  "seek file test3"
7  "read file test1"
8  "read file test2"
9  "read file test3"
10 "write file test1"
11 "write file test2"
12 "write file test3"
13 "write file test4"
14 "Успешно завершено тестов: " 13 / 13
15 "Все тесты завершены успешно"

```

Рисунок 4.9 – Результат интеграционного тестирования функций модуля «fs.lsp»

Результат интеграционного тестирования модуля «block.lsp» представлен на рисунке 4.10

```

1  "block set size test"
2  "block set offset test"
3  "block read test1"
4  "block read test2"
5  "block write test1"
6  "block write test2"
7  "get block pos test"
8  "get offset from pos test"
9  "Успешно завершено тестов: " 8 / 8
10 "Все тесты завершены успешно"

```

Рисунок 4.10 – Результат интеграционного тестирования функций модуля «block.lsp»

Результат модульного тестирования модуля «ata.lsp» представлен на рисунке 4.11

```

1  "ata test"
2  "Успешно завершено тестов: " 1 / 1
3  "Все тесты завершены успешно"

```

Рисунок 4.11 – Результат интеграционного тестирования функций модуля «ata.lsp»

4.10 Системное тестирование разработанной виртуальной файловой системы

Результат системного тестирования разработанной виртуальной файловой системы представлен на рисунке 4.12

```
1  > ls
2  ("boot" "fib.lsp")
3  > cd boot
4  > cwd
5  "/boot"
6  > touch test.lsp
7  > append test.lsp test-input
8  > cat test.lsp
9  "test-input"
10 > mkdir test-dir
11 > copy test.lsp test-dir/test2.lsp
12 > cat test-dir/test2.lsp
13 "test-input"
14 > rm test-dir/test2.lsp
15 > rmdir test-dir
16 >
```

Рисунок 4.12 – Результат системного тестирования разработанной виртуальной файловой системы

ЗАКЛЮЧЕНИЕ

Разработанная виртуальная файловая система представляет собой полностью функциональный слой абстракции, обеспечивающий прозрачное взаимодействие с данными независимо от их физической природы и местоположения. Унифицированный программный интерфейс, лежащий в основе системы, позволяет выполнять стандартный набор файловых операций над объектами, хранящимися в структурно различных источниках.

Основные результаты работы:

1. Изучены структуры файловых систем FAT32 и CDFS. Изучена структура блока параметров BIOS (BPB), структура FSInfo. Изучена таблица FAT, структура записи в каталоге. Изучены принципы работы с файлами и каталогами в файловой системе FAT32. Изучен формат данных Little-Endian и Big-Endian. Изучена структура дескрипторов томов, таблицы путей. Изучены принципы работы с файлами и каталогами в файловой системе CDFS.

2. Спроектирована виртуальная файловая система. Спроектированы классы абстрактной файловой системы и абстрактного класса. Спроектированы методы работы с файлом. Спроектированы вспомогательные функции взаимодействия с файловыми системами FAT32 и CDFS.

3. Спроектирован интерфейс виртуальной файловой системы. Спроектированы методы классов файловых систем FAT32 и CDFS.

4. Реализовал виртуальную файловую систему и её интерфейс. Реализованы методы работы с файлом.

5. Реализовал файловые системы FAT32 и CDFS. Реализована инициализация файловых систем. Реализованы методы классов файловых систем.

6. Реализовал командный интерпретатор. Реализованы команды для интерактивного взаимодействия с файловой системой.

Все требования, объявленные в техническом задании, были полностью реализованы, все задачи, поставленные в начале разработки проекта, были также решены.

Готовый рабочий проект представлен в виде библиотек. Библиотеки находятся в публичном доступе, поскольку опубликованы в репозитории «os-ri».

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Holm, N. M. LISP from Nothing / N. M. Holm. – Morrisville : Lulu Press, 2020. – 351 с. – ISBN 9781716508837. – Текст : непосредственный.
2. Friedman, D. P. The Little LISPer, Third Edition / D. P. Friedman. – Upper Saddle River : Pearson, 1989. – 222 с. – ISBN 9780023397639. – Текст : непосредственный.
3. Shangguan, Z. MODERN LISP SYSTEMS PROGRAMMING / Z. Shangguan. – [б. м.] : [б. и.], 2025. – 248 с. – ISBN 9788261924319. – Текст : непосредственный.
4. Graham, P. ANSI Common LISP / P. Graham. – Upper Saddle River : Pearson, 1995. – 448 с. – ISBN 9780133708752. – Текст : непосредственный.
5. Seibel, P. Practical Common Lisp / P. Seibel. – New York : Apress, 2012. – 500 с. – ISBN 9781430242901. – Текст : непосредственный.
6. Watson, M. Common LISP Modules / M. Watson. – New York : Springer, 1991. – 207 с. – ISBN 9780387976143. – Текст : непосредственный.
7. Touretzky, D. S. Common LISP: A Gentle Introduction to Symbolic Computation / D. S. Touretzky. – Mineola : Dover Publications, 2013. – 608 с. – ISBN 9780486498201. – Текст : непосредственный.
8. Levin, M. I. LISP 1.5 Programmer's Manual / M. I. Levin. – Cambridge : The MIT Press, 1962. – 112 с. – ISBN 9780262130110. – Текст : непосредственный.
9. de Groot, C. The Genius of Lisp / C. de Groot. – Ontario : Berksoft Publications, 2026. – 289 с. – ISBN 9781069886415. – Текст : непосредственный.
10. Keene, S. E. Object-Oriented Programming in COMMON LISP: A Programmer's Guide to CLOS / S. E. Keene. – Reading : Addison-Wesley Professional, 1989. – 288 с. – ISBN 9780201175899. – Текст : непосредственный.

11. Abelson, H. Structure and Interpretation of Computer Programs / H. Abelson, G. J. Sussman, J. Sussman. – Cambridge : The MIT Press, 1996. – 657 с. – ISBN 9780262011532. – Текст : непосредственный.
12. Barski, C. Land of Lisp: Learn to Program in Lisp, One Game at a Time! / C. Barski. – New York : Random House Publishing Services, 2010. – 800 с. – ISBN 9781593273491. – Текст : непосредственный.
13. Allen, J. Anatomy of Lisp / J. Allen. – New York : McGraw-Hill College, 1978. – 464 с. – ISBN 9780070011151. – Текст : непосредственный.
14. Pereira, E. A. The Lisp Handbook: From Syntax to Metaprogramming with Macros / E. A. Pereira. – [б. м.] : [б. и.], 2025. – 168 с. – ISBN 9788298155823. – Текст : непосредственный.
15. Holden, D. Build Your Own Lisp / D. Holden. – North Charleston : CreateSpace Independent Publishing Platform, 2014. – 213 с. – ISBN 9781501006623. – Текст : непосредственный.
16. Herda, M. The Common Lisp Condition System: Beyond Exception Handling with Control Flow Mechanisms / M. Herda. – New York : Apress, 2020. – 320 с. – ISBN 9781484261330. – Текст : непосредственный.
17. Weitz, E. Common Lisp Recipes: A Problem-Solution Approach / E. Weitz. – New York : Apress, 2015. – 771 с. – ISBN 9781484211779. – Текст : непосредственный.
18. Queinnec, C. Lisp in Small Pieces / C. Queinnec. – Cambridge : Cambridge University Press, 1996. – 534 с. – ISBN 9780521562478. – Текст : непосредственный.
19. Domkin, V. Programming Algorithms in Lisp: Writing Efficient Programs with Examples in ANSI Common Lisp / V. Domkin. – New York : Apress, 2021. – 392 с. – ISBN 9781484264270. – Текст : непосредственный.
20. Steele, G. Common LISP: The Language / G. Steele. – Amsterdam : Elsevier S & T, 1990. – 1056 с. – ISBN 9780080502267. – Текст : непосредственный.

21. Winston, P. H. Lisp / P. H. Winston, B. K. Horn. – Hoboken : Prentice Hall, 1988. – 611 с. – ISBN 9780201083194. – Текст : непосредственный.
22. Graham, P. On Lisp: Advanced Techniques for Common Lisp / P. Graham. – Hoboken : Prentice Hall, 1993. – 413 с. – ISBN 9780130305527. – Текст : непосредственный.
23. Jones, A. Advanced Techniques in Common LISP: Expert Insights and In-Depth Applications / A. Jones. – [б. м.] : [б. и.], 2024. – 306 с. – ISBN 9788345675014. – Текст : непосредственный.
24. Slade, S. Object-Oriented Common Lisp / S. Slade. – Englewood Cliffs : Prentice Hall, 1997. – 774 с. – ISBN 9780136059400. – Текст : непосредственный.
25. Yuasa, T. Introduction to Common Lisp / T. Yuasa, M. Hagiya. – Burlington : Morgan Kaufmann Publishers, 1987. – 293 с. – ISBN 9780127748603. – Текст : непосредственный.
26. Pereira, E. A. Mastering Common Lisp: Advanced Techniques for AI and Web Development / E. A. Pereira. – Seattle : Amazon Digital Services LLC - Kdp, 2025. – 119 с. – ISBN 9788298371063. – Текст : непосредственный.
27. Knott, G. D. Interpreting LISP: Programming and Data Structures / G. D. Knott. – New York : Apress, 2017. – 163 с. – ISBN 9781484227060. – Текст : непосредственный.
28. Brooks, R. A. Programming in Common LISP / R. A. Brooks. – New York : Wiley, 1985. – 303 с. – ISBN 9780471818885. – Текст : непосредственный.
29. Sangal, R. Programming Paradigms in Lisp / R. Sangal. – New York : McGraw-Hill College, 1991. – 384 с. – ISBN 9780070546660. – Текст : непосредственный.
30. Gabriel, R. P. Performance and Evaluation of Lisp Systems / R. P. Gabriel. – Cambridge : The MIT Press, 1985. – 285 с. – ISBN 9780262070935. – Текст : непосредственный.

31. Hoyte, D. Let Over Lambda 50 Years of Lisp / D. Hoyte. – Morrisville : Lulu.com, 2008. – 384 с. – ISBN 978-1-4357-1275-1. – Текст : непосредственный.
32. Toolan, F. File System Forensics / F. Toolan. – Hoboken : John Wiley & Sons, 2025. – 496 с. – ISBN 9781394289790. – Текст : непосредственный.
33. Tharp, A. L. File Organization and Processing / A. L. Tharp. – New York : Wiley, 1991. – 416 с. – ISBN 9780471605218. – Текст : непосредственный.
34. Lunt, B. D. Fysos The Virtual File System / B. D. Lunt. – North Charleston : CreateSpace Independent Publishing Platform, 2014. – 217 с. – ISBN 9781499164893. – Текст : непосредственный.
35. Bar, M. Linux File Systems / M. Bar. – New York : McGraw-Hill Osborne Media, 2001. – 512 с. – ISBN 9780072129557. – Текст : непосредственный.
36. Grosshans, D. File systems: Design and implementation / D. Grosshans. – Englewood Cliffs : Prentice Hall, 1986. – 496 с. – ISBN 9780133145687. – Текст : непосредственный.
37. Pansuriya, H. File Systems in Operating Systems Understanding File Systems in Operating Systems: A Comprehensive Overview / H. Pansuriya. – Seattle : Amazon Digital Services LLC – Kdp, 2024. – 338 с. – ISBN 9788874287085. – Текст : непосредственный.
38. Folk, M. J. File Structures / M. J. Folk, B. Zoellick. – Reading : Addison-Wesley, 1991. – 590 с. – ISBN 9780201557138. – Текст : непосредственный.
39. Harbron, T. R. File Systems: Structures and Algorithms / T. R. Harbron. – Englewood Cliffs : Prentice Hall, 1988. – 280 с. – ISBN 9780133147094. – Текст : непосредственный.
40. Giampaolo, D. Practical File System Design / D. Giampaolo. – Amsterdam : Elsevier Science, 1998. – 256 с. – ISBN 9781558604971. – Текст : непосредственный.
41. Blokdyk, G. Virtual File System a Complete Guide / G. Blokdyk. – Queensland : Emereo Pty Limited, 2019. – 302 с. – ISBN 9780655936459. – Текст : непосредственный.

42. Russell, J. ISO 9660 / J. Russell, R. Cohn. – Norderstedt : Books on Demand, 2012. – 160 с. – ISBN 9785511061733. – Текст : непосредственный.
43. Arpaci-Dusseau, R. H. Operating Systems Three Easy Pieces / R. H. Arpaci-Dusseau, A. C. Arpaci-Dusseau. – Madison : Arpaci-Dusseau Books, LLC, 2018. – 747 с. – ISBN 9781985086593. – Текст : непосредственный.
44. Stallings, W. Operating Systems Internals and Design Principles / W. Stallings. – Upper Saddle River : Pearson, 2018. – 800 с. – ISBN 9780134670959. – Текст : непосредственный.
45. Акинин, М. В. Системное программирование в Linux. Часть 2. Файловые системы / М. В. Акинин, Н. В. Акинина, С. В. Засорин. – Москва : КУРС, 2026. – 128 с. – ISBN 9785907064812. – Текст : непосредственный.
46. Aniche, M. Simple Object-Oriented Design Create Clean, Maintainable Applications / M. Aniche. – New York : Simon and Schuster, 2024. – 192 с. – ISBN 9781633437999. – Текст : непосредственный.
47. Wirfs-Brock, R. Designing Object-Oriented Software / R. Wirfs-Brock, B. Wilkerson, L. Wiener. – Upper Saddle River : Pearson, 1990. – 368 с. – ISBN 9780136298250. – Текст : непосредственный.
48. Richards, M. Fundamentals of Software Architecture A Modern Engineering Approach / M. Richards, N. Ford. – Sebastopol : O'Reilly Media, 2025. – 543 с. – ISBN 9781098175511. – Текст : непосредственный.
49. Martin, R. C. Clean Architecture A Craftsman's Guide to Software Structure and Design / R. C. Martin. – Upper Saddle River : Prentice Hall, 2018. – 432 с. – ISBN 9780134494166. – Текст : непосредственный.
50. Wiegers, K. E. Software Requirements / K. E. Wiegers, J. Beatty. – Redmond : Microsoft Press, 2013. – 672 с. – ISBN 9780735679665. – Текст : непосредственный.
51. Семакин, И. Г. Основы алгоритмизации и программирования / И. Г. Семакин, А. П. Шестаков. – Москва : Издательский центр «Академия», 2013. – 144 с. – ISBN 9785769594458. – Текст : непосредственный.

52. Белик, А. Г. Проектирование и архитектура программных систем / А. Г. Белик, В. Н. Цыганенко. – Омск : Издательство ОмГТУ, 2016. – 96 с. – ISBN 9785814922588. – Текст : непосредственный.

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.9.

Цель и задачи разработки

Цель настоящей работы – создание виртуальной файловой системы, а также изучение структуры FAT32 и CDFS.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучение структуры FAT32 и CDFS;
- проектирование виртуальной файловой системы;
- проектирование интерфейса виртуальной файловой системы;
- реализация виртуальной файловой системы и её интерфейса;
- реализация файловых систем FAT32 и CDFS;
- реализация командного интерпретатора.

ВКРБ 2206157.09.03.04.26.007			
Имя работы	Фамилия И. О.	Инициалы	Дата
Разработчик	Сидоров А.А.		
Проверен	Петров А.А.		
Цель и задачи разработки		Лист 2	Листов 3
Выпускная квалификационная работа бакалавра		03.04.10-216	

Рисунок А.2 – Цель и задачи разработки

Файловая система FAT32

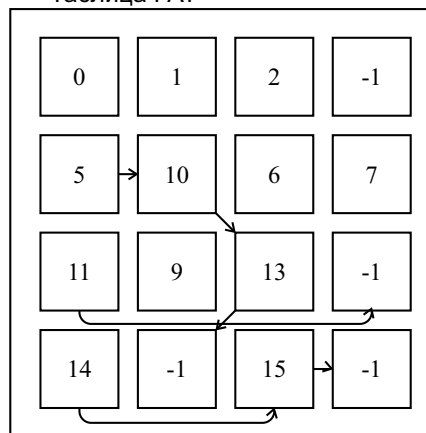
Номер сектора Значение

1	Блок параметров биос (BPB)
2	Структура FSINFO
3-6	4 резервных сектора
7	Копия блока параметров биос (BPB)
8	Копия структуры FSINFO
9-xx	Зарезервированные сектора
xx	Таблица FAT
xx	Копия таблицы FAT
xx	Область данных

Запись в каталоге

	Имя	Атрибуты	Время и дата создания	Ссылка на кластер	Размер
T	EXTTXT	0	11.01.20	4	1900
0xe5	DAT TXT	0	12.01.20	3	100
D	ATA2 C	readonly	12.01.20	12	1500
F	S DOC	system	13.01.20	8	768

Таблица FAT



ВКРБ 2206157.09.03.04.26.007			
Имя работы	Фамилия И. О.	Имя	Фамилия
Учебный номер	Степанов А.А.	Имя	Фамилия
Курс	Четвертый А.А.	Имя	Фамилия
Файловая система FAT32			
Выполнен и одобрен руководителем работ			
03.04.2016			

Рисунок А.3 – Файловая система FAT32

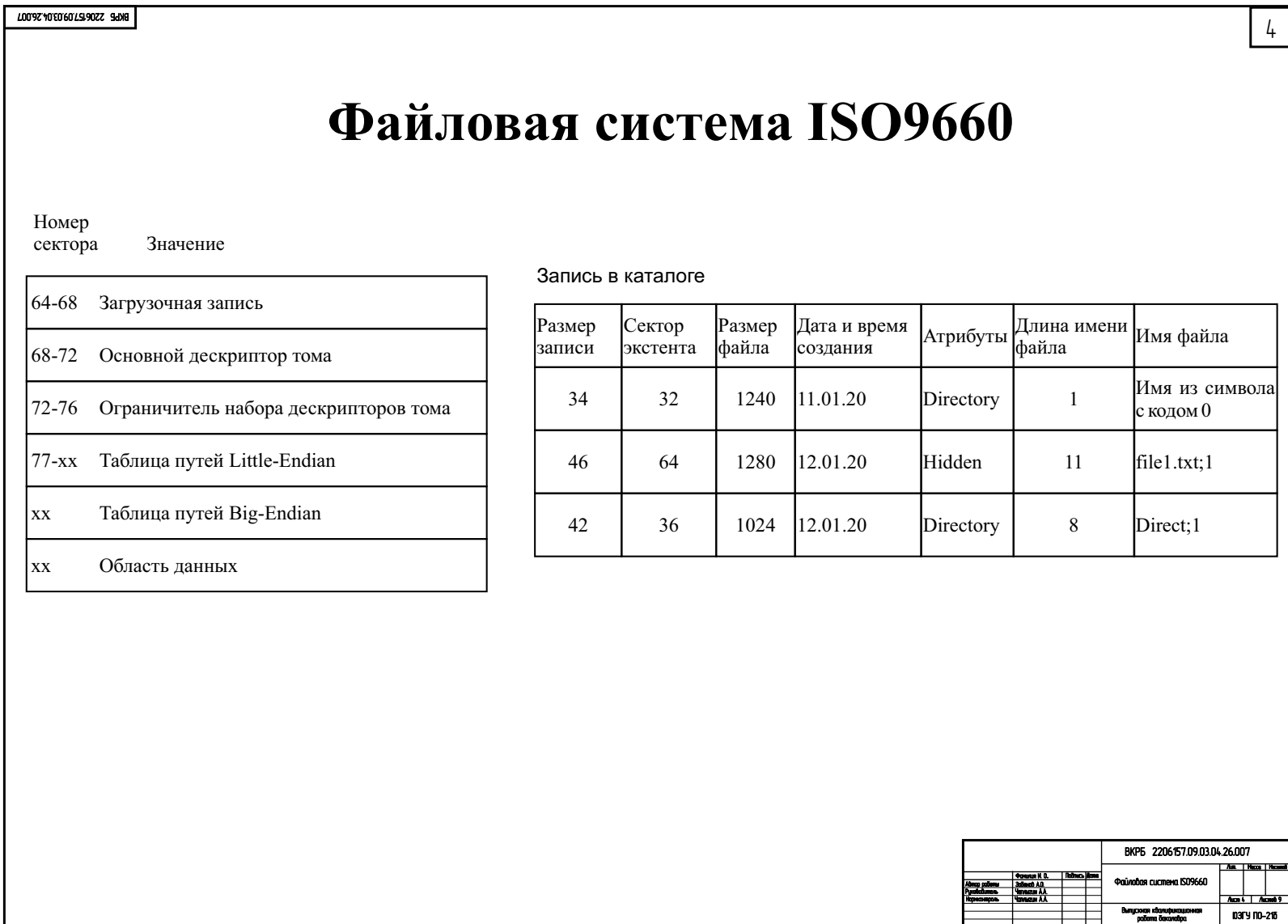


Рисунок А.4 – Файловая система ISO9660

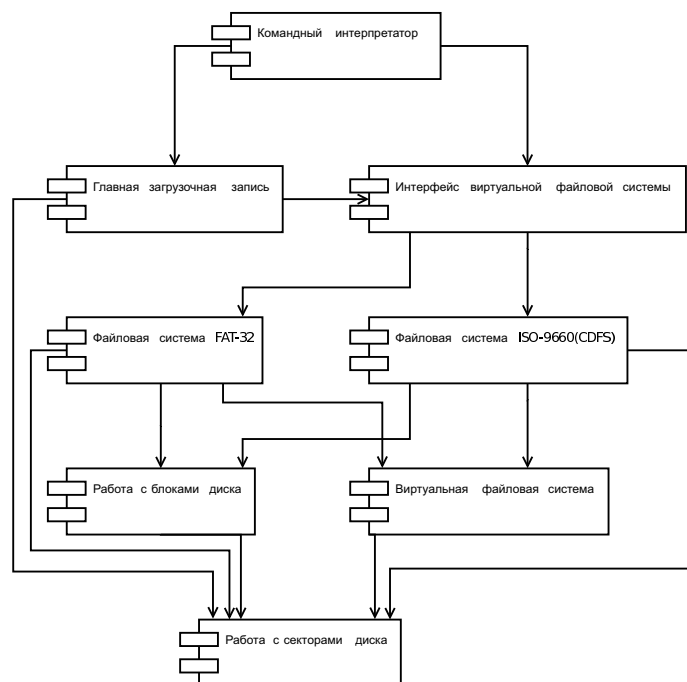
Интерфейс виртуальной файловой системы

Название функции	Аргументы	Описание функции
load-partition	disk-num part-num	Загрузить файловую систему из определённого раздела диска.
cur-dir	Аргументов нет	Получить путь текущего каталога. Возвращает полный путь текущего каталога в виде строки.
list-dir	path	Просмотреть содержимое каталога.
create-dir	path	Создать каталог.
remove-dir	path	Удалить каталог, если он пуст.
change-dir	path	Переместиться в каталог.
create-file	path	Создать файл.
remove-file	path	Удалить файл.
open-file	path	Открыть файл как поток для чтения и записи.
close-file	file-object	Закрыть файл.
read-file	file-object size	Прочитать определённое количество байт из потока файла, и переместить позицию внутри потока на то же количество.
write-file	file-object buffer	Записать массив байт в поток файла, и переместить позицию внутри потока на то же количество.
tell-file	file-object	Получить позицию внутри потока файла.
seek-file	file-object offset origin	Переместить позицию внутри потока файла.
is-directory	path	Проверить, является ли файл каталогом.
rename	path name	Переименовать файл или каталог.
free-space	Аргументов нет	Получить объём свободного места в текущем разделе диска.
set-attr	path new-attributes	Изменить атрибуты файла или каталога.
fstat	path	Получить метаданные файла или каталога.

ВРБ 2206157.09.03.04.26.007			
Имя работы	Фамилия И. О.	Имя	Фамилия
Учебный номер	Сидоров А.А.	Имя	Фамилия
Курс	Четвёртый А.А.	Имя	Фамилия
Интерфейс: ВРБ			
Выполнен информационный			
работы			
03.04.2016			

Рисунок А.5 – Интерфейс виртуальной файловой системы

Архитектура виртуальной файловой системы



Виртуальная файловая система – два класса (FileSystem и File), методы которого являются функциями интерфейса виртуальной файловой системы.

Поскольку функции open-file, close-file, read-file, write-file, seek-file, tell-file работают с файлами, то функция open-file будет возвращать файловый объект, а остальные будут работать с этим файловым объектом. Из-за этого стоит выделить класс файлового объекта (File).

Поскольку чтение и запись файла по блокам одинакова для всех файловых систем, то можно реализовать методы read-file, write-file, seek-file, tell-file, а в реализациях конкретных файловых систем вызывать методы родительского класса и менять метаданные файлов и каталогов.

Поле blocks в классе файла – список блоков файла.

Поле position в классе файла – пара из номера текущего блока и позиции в нём.

Если во время чтения позиция вышла за текущий блок, то проверяется есть ли у этого файла следующий блок, если блока нет, то вызывается ошибка, иначе номер следующего блока записывается в поле position, позиция в текущем блоке в поле position обнуляется.

При записи в файл может понадобиться добавление нового блока к файлу, для этого стоит добавить метод new-block, реализация которого будет в конкретной файловой системе.

Для остальных функций интерфейса виртуальной файловой выделяется класс файловой системы, реализация функций будет в конкретной файловой системе.

Для того чтобы раскрывать каталога в дереве файлов и каталогов добавляется метод load-dir, реализация которого будет в конкретной файловой системе.

Каждая файловая система – два класса, являющиеся наследниками классов виртуальной файловой системы (FileSystem и File), которые реализуют функции интерфейса виртуальной файловой системы.

ВКРБ 2206157.09.03.04.26.007			
Автор работы	Проверил И. А.	Подпись	Дата
Рецензент	Сметов А.А.		
Проверено	Чепелев А.А.		
Архитектура ВФС			
Выпускная квалификационная работа бакалавра			
03.04.2016			

Рисунок А.6 – Архитектура виртуальной файловой системы



Рисунок А.7 – Диаграмма классов

Тестирование

Результат интеграционного тестирования функций модуля "<vfs.lsp">

```
"join test"
"parse path test"
"load path test"
"get parent dir test"
"list dir test"
"create dir test"
"remove dir test"
"change dir test"
"create file test"
"remove file test"
"open file test"
"is directory test"
"rename test"
"free space test"
"fstat test"
"work with file test"
"set attributes test"
"Успешно завершено тестов: " 26 / 26
"Все тесты завершены успешно"
```

Результат системного тестирования разработанной виртуальной файловой системы

```
> ls
("boot" "fib.lsp")
> cd boot
> cwd
"/boot"
> touch test.lsp
> append test.lsp test-input
> cat test.lsp
"test-input"
> mkdir test-dir
> copy test.lsp test-dir/test2.lsp
> cat test-dir/test2.lsp
"test-input"
> rm test-dir/test2.lsp
> rmdir test-dir
>
```

ВКРБ 2206157.09.03.04.26.007			
Имя	Фамилия И. О.	Подпись	Дата
Исполнитель	Сидоров А.А.		
Проверенный	Петров А.А.		
Утвержденный	Сидоров А.А.		
Выполнен идентификационный		03.04.2016	
работы		ПО-216	

Рисунок А.8 – Тестирование

Рисунок А.9 – Заключение

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

fat32-fs.lsp

```
1 (defconst +fat32-fsinfo-eval+ 0xFFFFFFFF)
2
3 (defun parse-bios-parameter-block ()
4   "Парсер блока параметров BIOS"
5   (&&& bpb-struct-> (parse-struct '((jmp-command . 3)
6                                     (oem-id . 8)
7                                     (bytes-per-sector . 2) ;; must be 512
8                                     (sectors-per-block . 1) ;; for block size
9                                     (reserved-sectors-count . 2) ;;
10                                      boot-record + padding size in sectors
11                                      before FAT
12                                     (fat-tables-count . 1) ;; must be 2
13                                     (root-dir-entries-count . 2)
14                                     (volume-size-small . 2) ;; size of volume
15                                      in lba-sectors if less than 65536
16                                      else 0
17                                     (media-descriptor-type . 1)
18                                     (sectors-per-fat-fat16 . 2)
19                                     (sectors-per-track . 2)
20                                     (heads-count . 2)
21                                     (hidden-sectors-count . 4)
22                                     (volume-size-large . 4) ;; size of volume
23                                      in lba-sectors if more than 65535
24                                      else 0
25                                     (sectors-per-fat-fat32 . 4) ;; for fat
26                                      table size
27                                     (active-fat-struct . 2) ;; 4 bit for
28                                      active fat num + 3 bit reserved + 1
29                                      bit if fat copying + 8 bit reserved
30                                     (fat-version . 2)
31                                     (root-entry-first-block . 4) ;; for root
32                                      entry
33                                     (fsinfo-sector . 2) ;; fsinfo sector
34                                      number (must be before fat table)
35                                     (bpb-copy-sector . 2)
36                                     (reserved1 . 12)
37                                     (disk-type-num . 1)
38                                     (reserved2 . 1)
39                                     (volume-signature . 1) ;; 41(0x29) if
40                                      volume-serial or volume-label are not
41                                      zero else 40(0x28)
42                                     (volume-serial . 4)
43                                     (volume-label . 11)
44                                     (system-id . 8)
45                                     (reserved3 . 420)
46                                     (bpb-end-signature . 2)))) ;; must be
47                                     43605(aa55)
48   (return (progn
49     (unless (= 512 (arr-get-num (get-hash bpb-struct 'bytes-per-sector) 0 2))
```

```

36         (raise 'error "parse-bios-parameter-block: bytes per sector
           in bpb is not 512"))
37 (let ((hidden-sectors-count (arr-get-num (get-hash bpb-struct '
    hidden-sectors-count) 0 4)))
38     (let ((fat-tables-count (arr-get-num (get-hash bpb-struct '
    fat-tables-count) 0 1))
39 (active-fat-struct (arr-get-num (get-hash bpb-struct 'active-fat-struct
    ) 0 2)))
40         (FAT32FileSystem-set-fat-start-sectors *file-system* (
    make-array fat-tables-count))
41         (FAT32FileSystem-set-fat-sectors *file-system* (+
    hidden-sectors-count (arr-get-num (get-hash bpb-struct
    'sectors-per-fat-fat32) 0 4)))
42         (for i 0 fat-tables-count
43             (seta (FAT32FileSystem-fat-start-sectors *
    file-system*) i (+ hidden-sectors-count (+ (
    arr-get-num (get-hash bpb-struct '
    reserved-sectors-count) 0 2) (* i (
    FAT32FileSystem-fat-sectors *file-system*))))))
44         (FAT32FileSystem-set-fat-active-num *file-system* (&
    active-fat-struct 15))
45         (FAT32FileSystem-set-fat-copying *file-system* (if (= (
    get-bit-from-num active-fat-struct 7) 0) t nil)))
46         (set-block-size (arr-get-num (get-hash bpb-struct '
    sectors-per-block) 0 1))
47         (let ((volume-size-small (arr-get-num (get-hash bpb-struct
    'volume-size-small) 0 2))
48 (volume-size-large (arr-get-num (get-hash bpb-struct 'volume-size-large
    ) 0 4)))
49             (if (= volume-size-small 0) (
    FAT32FileSystem-set-volume-size *file-system*
    volume-size-large)
50 (FAT32FileSystem-set-volume-size *file-system* volume-size-small)))
51         (set-block-offset (- (+
52 (aref (FAT32FileSystem-fat-start-sectors *file-system*)
53         (- (array-size (
    FAT32FileSystem-fat-start-sectors
    *file-system*)) 1))
54 (FAT32FileSystem-fat-sectors *file-system*))
55         (* 2 *block-sectors*)))
56         (set-blocks-start-num 2)
57         (FAT32FileSystem-set-fsinfo-sector *file-system* (+
    hidden-sectors-count (arr-get-num (get-hash bpb-struct '
    fsinfo-sector) 0 2)))
58         (let ((root-entry (make-hash)))
59             (set-hash root-entry 'name 'root)
60             (set-hash root-entry 'size -1)
61             (set-hash root-entry 'first-block (arr-get-num (get-hash
    bpb-struct 'root-entry-first-block) 0 4))
62             (set-hash root-entry 'creation-date-time nil)
63             (set-hash root-entry 'modify-date-time nil)
64             (set-hash root-entry 'access-date nil)
65             (set-hash root-entry 'attributes '(DIRECTOR))
66             (set-hash root-entry 'parent-first-block nil))

```

```

67         (set-hash root-entry 'dir t)
68         (FAT32FileSystem-set-root-entry *file-system* root-entry)
69         )))))
70 (defun parse-fsinfo ()
71   "Парсер структуры FSInfo"
72   (&&& fsinfo-struct->(parse-struct '((struct-signature . 4) ;; 1096897106(0
73     x41615252)
74     (reserved1 . 480)
75     (check-signature . 4) ;; 1631679090(0x61417272)
76     (free-blocks-count . 4) ;; 4294967295(0xFFFFFFFF) если
77     посчитать
78     (free-block-num . 4) ;; 4294967295(0xFFFFFFFF) если посчитать
79     (reserved2 . 12)
80     (end-signature . 4))) ;; 43605(0xAA550000)
81   return (progn
82     (FAT32FileSystem-set-free-blocks-count *file-system* (arr-get-num (
83       get-hash fsinfo-struct 'free-blocks-count) 0 4))
84     (FAT32FileSystem-set-free-block-num *file-system* (arr-get-num (get-hash
85       fsinfo-struct 'free-block-num) 0 4))))))
86
87 (defmethod fs-init ((fat32fs FAT32FileSystem) disk start-sector sector-count)
88   "Загрузка файловой системы FAT32 с диска disk, начиная с сектора
89   start-sector, с количеством секторов sector-count. Чтение блока
90   параметров BIOS, структуры FSInfo, таблицы FAT."
91   (set-block-disk disk)
92   (funcall (parse-bios-parameter-block) (stream-from-arr (ata-read-sectors
93     disk start-sector 1) nil))
94   (when (> (FAT32FileSystem-volume-size fat32fs) sector-count) (raise 'error
95     "fs-init(fat32): volume size in bpb is higher than volume size in args")
96     )
97   (funcall (parse-fsinfo) (stream-from-arr (ata-read-sectors disk (
98     FAT32FileSystem-fsinfo-sector fat32fs) 1) nil))
99   (FAT32FileSystem-set-fat-table fat32fs (make-hash))
100   (let ((root-entry (FAT32FileSystem-root-entry fat32fs)))
101     (set-hash
102       root-entry
103       'size
104       0))
105   (setq *file-system* fat32fs)
106   (setq *root-directory* (FAT32FileSystem-root-entry *file-system*))
107   (setq *working-directory* *root-directory*)
108   (setq *working-path* '("root"))
109   fat32fs)
110
111 (defmethod load-dir ((fat32fs FAT32FileSystem) dir)
112   "Раскрыть и сохранить содержимое каталога dir"
113   (if (or (null (get-hash dir 'dir)) (not (equal (get-hash dir 'dir) t))) nil
114     (let ((blocks (get-fat-chain (get-hash dir 'first-block)))
115           (dir-list ())
116           (block-stream nil)
117           (lfn-name ""))
118       (while (not (null blocks))
119         (setq block-stream (stream-from-arr (block-read (car blocks)) nil))

```



```

110 (while (not (null block-stream))
111   (let ((next-byte (get-byte block-stream)))
112     (unless (null next-byte)
113       (case (car next-byte)
114         (+fat32-entry-deleted+ (setq block-stream (cdr (get-array
115           block-stream 32))))
116         (+fat32-entry-free+ (setq blocks '(nil) block-stream nil))
117         (otherwise
118          (let ((entry-attributes (aref (car (get-array block-stream
119            12)) 11)))
120            (if (= entry-attributes +fat32-lfn-attribute+)
121                (progn
122                  (let ((parse-res (funcall (parse-lfn-entry)
123                    block-stream)))
124                    (setq lfn-name (concat (car parse-res) lfn-name)
125                      )
126                    (setq block-stream (cdr parse-res))))
127                  (progn
128                    (let ((parse-res (funcall (parse-fat32-dir-entry)
129                      block-stream)))
130                      (set-hash (car parse-res) 'parent-first-block (
131                        car (get-fat-chain (get-hash dir 'first-block
132                          ))))
133                      (unless (equal lfn-name "")
134                        (set-hash (car parse-res) 'name lfn-name))
135                      (setq lfn-name ""))
136                      (unless (fat32-is-special-name (get-hash (car
137                        parse-res) 'name))
138                        (setq dir-list (append dir-list (list (car
139                          parse-res)))))
140                      (setq block-stream (cdr parse-res))))))))))
141   (setq blocks (cdr blocks)))
142 (set-hash dir 'dir (list dir-list))))
143
144 (defmethod fstat* ((fat32fs FAT32FileSystem) file-hash)
145   "Получение метаданных файлового объекта file-hash"
146   (let ((stat (make-hash)))
147     (set-hash stat 'name (get-hash file-hash 'name))
148     (set-hash stat 'size (if (not (null (get-hash file-hash 'dir))) 0 (
149       get-hash file-hash 'size)))
150     (set-hash stat 'create-date (list (nth (get-hash file-hash '
151       creation-date-time) 2)
152                                         (nth (get-hash file-hash '
153       creation-date-time) 1)
154                                         (nth (get-hash file-hash '
155       creation-date-time) 0)))
156     (set-hash stat 'create-time (list (nth (get-hash file-hash '
157       creation-date-time) 3)
158                                         (nth (get-hash file-hash '
159       creation-date-time) 4)
160                                         (nth (get-hash file-hash '
161       creation-date-time) 5)))
162     (set-hash stat 'modify-date (list (nth (get-hash file-hash '
163       modify-date-time) 2)

```

```

147         (nth (get-hash file-hash '
148             modify-date-time) 1)
149         (nth (get-hash file-hash '
150             modify-date-time) 0)))
151     (set-hash stat 'modify-time (list (nth (get-hash file-hash '
152         modify-date-time) 3)
153         (nth (get-hash file-hash '
154             modify-date-time) 4)
155         (nth (get-hash file-hash '
156             modify-date-time) 5)))
157     (set-hash stat 'access-date (list (nth (get-hash file-hash 'access-date)
158         2)
159         (nth (get-hash file-hash 'access-date)
160             1)
161         (nth (get-hash file-hash 'access-date)
162             0)))
163     (set-hash stat 'access-time nil)
164     (set-hash stat 'isdir (if (null (get-hash file-hash 'dir)) nil t))
165     (set-hash stat 'create-time-ms (nth (get-hash file-hash '
166         creation-date-time) 6))
167     (set-hash stat 'flags (get-hash file-hash 'attributes))
168     stat))
169 (defmethod open-file* ((fat32fs FAT32FileSystem) file-hash)
170     "Открыть файл file-hash в каталоге dir как поток для чтения и записи.
171     Изменить время последнего доступа"
172     (when (not (null (get-hash file-hash 'dir))) (raise 'not-file "open-file:
173         directories cannot be opened"))
174     (let ((file (make-instance 'FAT32File)))
175         (FAT32File-set-name file (get-hash file-hash 'name))
176         (FAT32File-set-size file (get-hash file-hash 'size))
177         (FAT32File-set-position file '(0 . 0))
178         (FAT32File-set-blocks file (get-fat-chain (get-hash file-hash '
179             first-block)))
180         (FAT32File-set-dir-entry file file-hash)
181         (let ((time-list (get-cur-time))
182             (access-date nil))
183             (setq access-date (list (nth time-list 0) (nth time-list 1) (nth
184                 time-list 2)))
185             (fat32-update-entry file-hash `((access-date . ,access-date))))
186         file))
187 (defun update-fsinfo (free-count free-num)
188     "Обновить значения структуры fsinfo на диске"
189     (let ((fsinfo-bytes (make-array 512)))
190         (arr-set-num fsinfo-bytes 0 0x41615252 4)
191         (for i 4 484
192             (seta fsinfo-bytes i 0))
193         (arr-set-num fsinfo-bytes 484 0x61417272 4)
194         (arr-set-num fsinfo-bytes 488 free-count 4)
195         (arr-set-num fsinfo-bytes 492 free-num 4)
196         (for i 496 508
197             (seta fsinfo-bytes i 0))
198         (arr-set-num fsinfo-bytes 508 0xAA550000 4))

```

```

188 (ata-write-sectors *disk* (FAT32FileSystem-fsinfo-sector *file-system*) 1
    fsinfo-bytes)
189 (FAT32FileSystem-set-free-blocks-count *file-system* free-count)
190 (FAT32FileSystem-set-free-block-num *file-system* free-num)))
191
192 (defmethod new-block ((fat32fs FAT32FileSystem) file-object)
193   "Выделить новый блок для файла file-object"
194   (let ((fat-table (make-array (/ (* (FAT32FileSystem-fat-sectors fat32fs) +
    sector-size) 4)))
195     (fat-read-sectors (make-array (FAT32FileSystem-fat-sectors fat32fs)))
196     (cur-table-elem 0)
197     (free-count (FAT32FileSystem-free-blocks-count fat32fs))
198     (free-num (FAT32FileSystem-free-block-num fat32fs)))
199     (for i 0 (array-size fat-read-sectors) (seta fat-read-sectors i nil))
200     (when (= free-count +fat32-fsinfo-eval+)
201       (setq free-count 0)
202       (for i +fat-min-elem+ (array-size fat-table)
203         (fat-check-for-read fat-table fat-read-sectors i)
204         (when (= (aref fat-table i) +fat-block-free+)
205           (incf free-count))))
206     (when (= free-num +fat32-fsinfo-eval+)
207       (for i +fat-min-elem+ (array-size fat-table)
208         (fat-check-for-read fat-table fat-read-sectors i)
209         (when (= (aref fat-table i) +fat-block-free+)
210           (setq free-num (- i 1))
211           (setq i (array-size fat-table)))))
212     (when (not (null file-object))
213       (let ((free-block (get-free-block fat-table fat-read-sectors free-num))
214             (first-block (get-hash file-object 'first-block)))
215         (let ((empty-block (make-array *block-size*)))
216           (for i 0 *block-size* (seta empty-block i 0))
217           (block-write free-block empty-block))
218         (if (null first-block)
219             (progn
220               (set-hash file-object 'first-block free-block)
221               (update-fat-elem free-block +fat-block-end+))
222             (append-fat-chain first-block free-block))
223         (decf free-count)
224         (setq free-num free-block)))
225     (when (or
226           (≠ free-count (FAT32FileSystem-free-blocks-count fat32fs))
227           (≠ free-num (FAT32FileSystem-free-block-num fat32fs)))
228       (update-fsinfo free-count free-num))
229     nil))
230
231 (defmethod rename* ((fat32fs FAT32FileSystem) entry new-name)
232   "Переименовать файл или каталог entry на new-name"
233   (fat32-update-entry entry `((name . ,new-name))))
234
235 (defmethod remove-file* ((fat32fs FAT32FileSystem) dir entry)
236   "Пометить файл entry на удаление"
237   (when (not (null (get-hash entry 'dir)))
238     (raise 'not-file "remove-file: entry is not a file"))
239   (fat32-mark-for-delete entry))

```

```

240 (set-hash dir 'dir (list (filter #'(lambda (elem) (not (equal entry elem)))
    (car (get-hash dir 'dir)))))
241
242 (defmethod remove-dir* ((fat32fs FAT32FileSystem) parent-dir dir)
243   "Пометить каталог entry на удаление если он пустой"
244   (when (null (get-hash dir 'dir))
245     (raise 'not-directory "remove-dir: entry is not a dir"))
246   (when (not (null (car (get-hash dir 'dir))))
247     (raise 'not-empty-dir "remove-dir: dir is not empty"))
248   (fat32-mark-for-delete dir)
249   (set-hash parent-dir 'dir (list (filter #'(lambda (elem) (not (equal dir
    elem))) (car (get-hash parent-dir 'dir)))))
250
251 (defmethod free-space* ((fat32fs FAT32FileSystem))
252   "Получить объём свободного места в текущем разделе в байтах"
253   (let ((free-count (FAT32FileSystem-free-blocks-count fat32fs)))
254     (when (= free-count +fat32-fsinfo-eval+)
255       (new-block fat32fs nil)
256       (setq free-count (FAT32FileSystem-free-blocks-count fat32fs)))
257     (* free-count *block-size*))
258
259 (defmethod create-file* ((fat32fs FAT32FileSystem) dir name)
260   "Создать файл с названием name в каталоге dir"
261   (when (fat32-is-name-duplicate dir name)
262     (raise 'file-exists "create-file: file exists"))
263   (let ((new-entry (make-hash))
264         (date-time (get-cur-time)))
265     (set-hash new-entry 'name name)
266     (set-hash new-entry 'size 0)
267     (set-hash new-entry 'first-block ())
268     (new-block fat32fs new-entry)
269     (set-hash new-entry 'creation-date-time (append date-time (list 0)))
270     (set-hash new-entry 'modify-date-time date-time)
271     (set-hash new-entry 'access-date (list (nth date-time 0) (nth date-time
    1) (nth date-time 2)))
272     (set-hash new-entry 'attributes ())
273     (set-hash new-entry 'parent-first-block (get-hash dir 'first-block))
274     (set-hash new-entry 'short-name (fat32-generate-short-name dir name))
275     (set-hash new-entry 'dir nil)
276     (fat32-add-entry new-entry)
277     (set-hash dir 'dir
278       (list (append (car (get-hash dir 'dir)) (list new-entry))))
279     new-entry))
280
281 (defmethod create-dir* ((fat32fs FAT32FileSystem) dir name)
282   "Создать каталог с названием name в каталоге dir"
283   (when (fat32-is-name-duplicate dir name)
284     (raise 'file-exists "create-file: directory exists"))
285   (let ((new-entry (make-hash))
286         (date-time (get-cur-time)))
287     (set-hash new-entry 'name (copy-tree name))
288     (set-hash new-entry 'size 0)
289     (set-hash new-entry 'first-block ())
290     (new-block fat32fs new-entry)

```

```

291 (set-hash new-entry 'creation-date-time (append date-time (list 0)))
292 (set-hash new-entry 'modify-date-time date-time)
293 (set-hash new-entry 'access-date (list (nth date-time 0) (nth date-time
294 1) (nth date-time 2)))
295 (set-hash new-entry 'attributes '(DIRECTORY))
296 (set-hash new-entry 'parent-first-block (get-hash dir 'first-block))
297 (set-hash new-entry 'short-name (fat32-generate-short-name dir name))
298 (set-hash new-entry 'dir t)
299 (fat32-add-entry new-entry)
300 (set-hash dir 'dir
301 (list (append (car (get-hash dir 'dir)) (list new-entry))))
302 (fat32-create-special-entries new-entry)
303 new-entry))
304 (defmethod set-attr* ((fat32fs FAT32FileSystem) entry new-attributes)
305 "Изменить атрибуты файла entry на new-attributes"
306 (let ((attributes (fat32-sort-attributes new-attributes (not (null (
307 get-hash entry 'dir))))))
308 (fat32-update-entry entry `((attributes . ,attributes)))))

```

iso9660-fs.lsp

```

1 (defconst +boot-record-num+ 0) ;; Номер дескриптора типа Boot Record
2 (defconst +primary-desc-num+ 1) ;; Номер дескриптора типа Primary Volume
   Descriptor
3 (defconst +terminator-desc-num+ 255) ;; Номер дескриптора типа Volume
   Descriptor Set Terminator
4 (defconst +descriptor-sector+ 64) ;; Сектор с которого начинаются дескрипторы
5 (defconst +descriptor-size+ 4) ;; Размер дескриптора в секторах
6
7 ;; Класс файловой системы ISO9660
8 (defclass CDFSFileSystem FileSystem (fs-block-count path-table-size
   path-table-sector root-entry))
9
10 (defun parse-boot-record ()
11 "Парсер оставшейся части структуры дескриптора типа Boot Record"
12 (parse-struct '((boot-sys-id . 32)
13 (boot-id . 32)
14 (boot-use . 1977))))
15
16 (defun parse-primary-desc ()
17 "Парсер оставшейся части структуры дескриптора типа Primary Volume
   Descriptor"
18 (parse-struct '((unused1 . 1)
19 (sys-id . 32)
20 (volume-id . 32)
21 (unused2 . 8)
22 (volume-size-lsb . 4)
23 (volume-size-msb . 4)
24 (unused3 . 32)
25 (volume-set-size . 4)
26 (volume-seq-num . 4)
27 (block-size-lsb . 2)
28 (block-size-msb . 2)
29 (path-table-size-lsb . 4)

```

```

30      (path-table-size-msb . 4)
31      (path-table-sector-lsb . 4)
32      (path-table-copy-sector-lsb . 4)
33      (path-table-sector-msb . 4)
34      (path-table-copy-sector-msb . 4)
35      (root-dir-entry . 34)
36      (volume-set-id . 128)
37      (published-id . 128)
38      (data-prep-id . 128)
39      (app-id . 128)
40      (copyright-file-id . 37)
41      (abstract-file-id . 37)
42      (bibliographic-file-id . 37)
43      (volume-create-date-time . 17)
44      (volume-modification-date-time . 17)
45      (volume-expiration-date-time . 17)
46      (volume-effective-date-time . 17)
47      (file-struct-version . 1)
48      (unused4 . 1)
49      (app-used . 512)
50      (reserve . 653))))
51
52 (defun parse-set-terminator-desc ()
53   "Парсер оставшейся части структуры дескриптора типа Volume Descriptor Set Terminator"
54   (parse-struct '((reserve . 2041))))
55
56 (defun parse-descriptor-data (desc-type)
57   "Возврат парсера оставшейся части структуры дескриптора типа desc-type"
58   (case desc-type
59     (+boot-record-num+ (parse-boot-record))
60     (+primary-desc-num+ (parse-primary-desc))
61     (+terminator-desc-num+ (parse-set-terminator-desc))))
62
63 (defun parse-date-time-entry ()
64   "Парсер структуры даты и времени в записи в каталоге"
65   (&&& date-time-entry->(parse-struct '((year . 1)
66                                           (month . 1)
67                                           (day . 1)
68                                           (hour . 1)
69                                           (minute . 1)
70                                           (second . 1)
71                                           (offset . 1))))
72   (return (list (+ (arr-get-num (get-hash date-time-entry 'year) 0 1)
73                     1900)
74                (arr-get-num (get-hash date-time-entry 'month) 0 1)
75                (arr-get-num (get-hash date-time-entry 'day) 0 1)
76                (arr-get-num (get-hash date-time-entry 'hour) 0 1)
77                (arr-get-num (get-hash date-time-entry 'minute) 0 1)
78                (arr-get-num (get-hash date-time-entry 'second) 0 1))))
79
80 (defun cdfs-get-date-time-entry (date-time-bytes)
81   "Получить список даты и времени из записи в каталоге из массива байт date-time-bytes(размер массива 7)"

```

```

81 (car (funcall (parse-date-time-entry) (stream-from-arr date-time-bytes nil)
82 )))
83 (defun cdfs-get-attributes-entry (attr-byte)
84   "Получить список атрибутов из записи в каталоге из байта attr-byte"
85   (let ((attributes nil)
86         (all-attributes #(HIDDEN DIRECTORY ASSOCIATIVE FORMAT-IN-EXT-ATTR
87                           PERMISSIONS-IN-EXT-ATTR nil nil NOT-LAST-ENTRY)))
87     (for i 0 8
88       (unless (or (= i 5) (= i 6))
89         (unless (= 0 (& attr-byte (expt 2 i)))
90           (setq attributes (cons (aref all-attributes i) attributes))))))
91     attributes))
92
93 (defun parse-cdfs-dir-entry ()
94   "Парсер структуры записи в каталоге"
95   (&&& dir-entry1->(parse-struct '((entry-len . 1)
96                                     (entry-ext-attr-len . 1)
97                                     (extent-block-num-lsb . 4)
98                                     (extent-block-num-msb . 4)
99                                     (extent-size-lsb . 4)
100                                    (extent-size-msb . 4)
101                                    (entry-creation-date-time . 7)
102                                    (entry-file-flags . 1)
103                                    (interleaved-unit-size . 1)
104                                    (interleaved-gap-size . 1)
105                                    (volume-seq-num . 4)
106                                    (file-name-len . 1)))
107     dir-entry2->(parse-struct `((file-name . ,(arr-get-num (get-hash
108                                                              dir-entry1 'file-name-len) 0 1))
109                               (entry-padding . ,(if (= (% (arr-get-num (
110                                                                 get-hash dir-entry1 'file-name-len) 0
111                                                                 1) 2) 0) 1 0))))))
112   return (let ((dir-entry (make-hash)))
113     (set-hash dir-entry 'name (car (split #\; (arr-get-str (
114       get-hash dir-entry2 'file-name) 0 (array-size (get-hash
115       dir-entry2 'file-name))))))
116     (set-hash dir-entry 'size (arr-get-num (get-hash dir-entry1 '
117       extent-size-lsb) 0 4))
118     (set-hash dir-entry 'blocks (list (arr-get-num (get-hash
119       dir-entry1 'extent-block-num-lsb) 0 4)))
120     (let ((left-size (get-hash dir-entry 'size))
121           (cur-block (car (get-hash dir-entry 'blocks))))
122       (while (> left-size *block-size*)
123         (setq left-size (- left-size *block-size*)
124               cur-block (+ cur-block 1))
125         (set-hash dir-entry 'blocks (append (get-hash dir-entry '
126           blocks) (list cur-block))))))
127     (set-hash dir-entry 'creation-date-time (
128       cdfs-get-date-time-entry (get-hash dir-entry1 '
129       entry-creation-date-time)))
130     (set-hash dir-entry 'attributes (cdfs-get-attributes-entry (
131       arr-get-num (get-hash dir-entry1 'entry-file-flags) 0 1)))

```

```

121         (set-hash dir-entry 'dir (contains (get-hash dir-entry '
122             attributes) 'DIRECTORY))
123         dir-entry)))
124 (defun parse-descriptor (cdsfs)
125     "Парсер структура основы дескриптора"
126     (&&&
127         descriptor1->(parse-struct '((desc-type . 1)
128             (desc-id . 5)
129             (desc-ver . 1)))
130         descriptor2->(parse-descriptor-data (arr-get-num (get-hash descriptor1 '
131             desc-type) 0 1))
132         return (case (arr-get-num (get-hash descriptor1 'desc-type) 0 1)
133             (+boot-record-num+ t)
134             (+terminator-desc-num+ nil)
135             (+primary-desc-num+ (progn
136                 (set-block-offset 0)
137                 (set-block-size (/ (arr-get-num (get-hash
138                     descriptor2 'block-size-lsb) 0 2) +
139                     sector-size+))
140                 (CDFSFileSystem-set-fs-block-count
141                     cdsfs
142                     (arr-get-num (get-hash descriptor2 '
143                         volume-size-lsb) 0 4))
144                 (CDFSFileSystem-set-path-table-size
145                     cdsfs
146                     (arr-get-num (get-hash descriptor2 '
147                         path-table-size-lsb) 0 4))
148                 (CDFSFileSystem-set-path-table-sector
149                     cdsfs
150                     (arr-get-num (get-hash descriptor2 '
151                         path-table-sector-lsb) 0 4))
152                 (CDFSFileSystem-set-root-entry
153                     cdsfs
154                     (car (funcall (parse-cdfs-dir-entry)
155                         (stream-from-arr (get-hash descriptor2 'root-dir-entry) nil))))
156                     (set-hash (CDFSFileSystem-root-entry cdsfs)
157                         'name "root")
158                     t))
159             (otherwise (raise 'error "parse-descriptor: unknown descriptor"))
160             )))
161 (defmethod fs-init ((self CDFSFileSystem) disk start-sector sector-count)
162     "Загрузка файловой системы ISO9660 с диска disk, начиная с сектора
163         start-sector, с количеством секторов sector-count. Чтение основного
164         дескриптора тома, записи в каталоге для корневого каталога из основного
165         дескриптора тома."
166     (set-block-disk disk)
167     (let ((not-terminator t)
168         (descriptor-sector (+ start-sector +descriptor-sector+)))
169         (while (not (null not-terminator))
170             (setq not-terminator
171                 (car (funcall (parse-descriptor self)

```



```

162         (stream-from-arr (ata-read-sectors disk descriptor-sector +
163             descriptor-size+) nil)))
164         descriptor-sector (+ descriptor-sector +descriptor-size+)))
165     (setq *file-system* self)
166     (setq *root-directory* (CDFFileSystem-root-entry *file-system*))
167     (setq *working-directory* *root-directory*)
168     (setq *working-path* '("root"))
169     self))
170 (defmethod load-dir ((self CDFFileSystem) dir)
171     "Раскрыть и сохранить содержимое каталога dir"
172     (if (or (null (get-hash dir 'dir)) (not (equal (get-hash dir 'dir) t))) nil
173         (let ((left-size (get-hash dir 'size))
174             (blocks (get-hash dir 'blocks))
175             (dir-list ())
176             (block-stream nil))
177             (while (and (not (null blocks)) (> left-size 0))
178                 (setq block-stream (stream-from-arr (block-read (car blocks)) nil))
179                 (while (and (not (null block-stream)) (> left-size 0))
180                     (let ((next-byte (get-byte block-stream)))
181                         (if (or (null next-byte) (= (car next-byte) 0)) (setq
182                             block-stream ()
183                             left-size (- left-size (- *block-size* (AStream-byte-num
184                                 block-stream)))))
185                             (progn (setq left-size (- left-size (car next-byte)))
186                                 (when (>= left-size 0)
187                                     (let ((parse-res (funcall (parse-cdfs-dir-entry)
188                                         block-stream)))
189                                         (setq dir-list (append dir-list (list (car
190                                             parse-res))))
191                                         block-stream (cdr parse-res))))))))
192                 (setq blocks (cdr blocks)))
193             (set-hash dir 'dir (list dir-list))))))
194 (defmethod fstat* ((self CDFFileSystem) file-hash)
195     "Получение метаданных файлового объекта file-hash"
196     (let ((stat (make-hash)))
197         (set-hash stat 'name (get-hash file-hash 'name))
198         (set-hash stat 'size (if (get-hash file-hash 'dir) 0 (get-hash file-hash
199             'size)))
200         (set-hash stat 'create-date (list (nth (get-hash file-hash '
201             creation-date-time) 2)
202             (nth (get-hash file-hash '
203                 creation-date-time) 1)
204             (nth (get-hash file-hash '
205                 creation-date-time) 0)))
206         (set-hash stat 'create-time (list (nth (get-hash file-hash '
207             creation-date-time) 3)
208             (nth (get-hash file-hash '
209                 creation-date-time) 4)
210             (nth (get-hash file-hash '
211                 creation-date-time) 5)))
212         (set-hash stat 'modify-date nil)
213         (set-hash stat 'modify-time nil))

```

```

204 (set-hash stat 'access-date nil)
205 (set-hash stat 'access-time nil)
206 (set-hash stat 'isdir (if (null (get-hash file-hash 'dir)) nil t))
207 (set-hash stat 'flags (get-hash file-hash 'attributes))
208 stat))
209
210 (defmethod open-file* ((self CDFSFileSystem) file-hash)
211   "Открыть файл как поток для чтения"
212   (when (not (null (get-hash file-hash 'dir))) (raise 'not-file "open-file:
    directories cannot be opened"))
213   (let ((file (make-instance 'CDFSFile)))
214     (CDFSFile-set-name file (get-hash file-hash 'name))
215     (CDFSFile-set-size file (get-hash file-hash 'size))
216     (CDFSFile-set-position file '(0 . 0))
217     (CDFSFile-set-blocks file (get-hash file-hash 'blocks))
218     file))

```

Автор ВКР

(подпись, дата)

А. О. Забанов

Руководитель ВКР

(подпись, дата)

А. А. Чаплыгин

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

Место для диска